

Unicode

Az Unicode univerzális karakterkódolási szabvány írott karakterekhez és szöveghez. Lefedi a világ összes modern és ősi nyelvének összes karakterét. Tartalmaz továbbá műszaki szimbólumokat, írásjeleket és sok más olyan karaktert, melyet írott szövegben használnak. Az Unicode széles körben használt és támogatott karakterkódolási szabvány.

Alapvető fogalmak

Kódtér: A karaktereket kódoló egész számok tartománya

Kódpont: A kódtér egy eleme, egy karaktert kódoló egész szám.

Basic Multilingual Plane (BMP): Az első 65 536 kódpontot (U+0000-U+FFFF) tartalmazó sík (0. sík). A BMP-be tartozik a Unicode karakterek többsége szinte minden szöveges adat esetén. \

Karakterkódolások

A szabvány által megadott karakterkódolások

Mindhárom karakterkódolással ábrázolható valamennyi Unicode karakter. Az UTF a Unicode transzformációs formátum (Unicode transformation format) rövidítése.

UTF-8

A kódpontok ábrázolása 1-4 byte-on történik (változó szélességű kódolás).

- Az U+0000-U+007F tartományba eső kódpontokat egy byte ábrázolja (128 ASCII karakter).
- Az U+0080-U+07FF tartományba eső kódpontokat 2 byte ábrázolja
- A BMP összes további kódpontját 3 byte ábrázolja.
- A BMP-n kívüli kódpontokat 4 byte ábrázolja.

Egy kódpontot ábrázoló byte-sorozat első byte-ja meghatározza a sorozat hosszát (egyszerű feldolgozhatóság). A használt byte-ok számának tekintetében a legtömörebb kódolás. Kevésbé hatékony kelet-ázsiai írásrendszerek esetén, mint például a kínai, japán és koreai.

UTF-16

Minden kódpont ábrázolása 2 vagy 4 byte-on történik (változó szélességű kódolás).

- A BMP karaktereinek ábrázolása 2 byte-on, az összes többi pedig 4 byte-on.
- A BMP karaktereinek ábrázolására van optimalizálva.
- A BMP kódpontjainál rögzített szélességű kódolási formaként kezelhető.

Kompromisszumot képvisel a hatékony hozzáférés és a hatékony tárháshasználat között.

UTF-32

Minden kódpont ábrázolása 4 byte-on történik (rögzített szélességű kódolás). Ez a legegyszerűbb karakterkódolás. Feldolgozás szempontjából ez a leghatékonyabb, azonban tárolás szempontjából a legkevésbé hatékony.

További karakterkódolás

ISO/IEC 8859-2 (Latin-2)

A közép-európai nyelvekhez. Az albán, bosnyák, cseh, horvát, lengyel, magyar, német, román, szerb (latin betűs írás), szlovák, szlovén, szorb nyelvekhez alkalmas.

Unicode karakterek kifejezése

CSS

Unicode karakterek megadásához használhatunk `\hhhhh` formájú vezérlősorozatokat, ahol hhhhh a Unicode karakter kódpontját ábrázoló legalább egy és legfeljebb 6 karakterből álló hexadecimális számjegysorozat.

```
\9, \09, \000009 - vízszintes tabulátor karaktert jeleníti meg  
\A9, \0A9, \0000A9 - a copyright szimbólumot jelenti
```

JSON

Sztringekben a BMP-hez tartozó Unicode karakterek megadhatóak `\uhhhh` formájú vezérlősorozatokkal, ahol hhhh a kódpontot ábrázoló négy hexadecimális számjegy.

```
\u00A9, \u262F
```

XML

Szövegben, attribútumértékekben és literális egyed értékekben Unicode karakterek kifejezhetők az alábbi formájú karakterhivatkozásokkal:

```
&#nnnn: ahol nnnn a kódpontot ábrázoló decimális számjegysorozat  
&#169, &#9775, &#128570
```

HTML

Számos Unicode karakter fejezhető ki **&név;** formájú nevesített karakterhivatkozásokkal.

```
&Eacute; (U+00C9 = É)  
&eacute; (U+00E9 = é)  
&star; (U+2606 = csillag_rajz)
```

Média Típusok

Internet protokollokon keresztül továbbított tartalmak formátumának jelzésére szolgálnak. Egy felső-szintű típusból (top-level type) és egy altípusból (subtype) állnak. Lehetnek paramétereik is. (például: text/html; charset=UTF-8)

Média típusok komponensei

Felső szintű típus

A felső szintű típusok nevei kisbetű-nagybetű érzéketlenek.

Altípus

Az altípusok nevei kisbetű-nagybetű érzéketlenek. Altípus regisztrálása egy adott fába történik a használat körétől függően. **Az altípus neve hordozza, hogy melyik regisztrációs fába tartozik.** Az altípus nevének elején a fa. előtag határozza meg a regisztrációs fát.

Paraméterek

A paraméterek nevei kisbetű-nagybetű érzéketlenek. Tetszőleges sorrendben adhatóak meg, azonban minden paraméter **legfeljebb csak egyszer**. A paraméterek értékeire nincs előre írt szintaxis.

A média típus regisztráció határozza meg a paraméterek neveit és lehetséges értékeit, valamint az egyes paraméterek jelentését.

- Content-Type: text/html; charset = UTF-8
- Content-Type: audio/ogg; codecs = vorbis
- Content-Type: video/ogg; codecs = "theora, vorbis"
- Content-Type: multipart/form-data; boundary = AaB03x
- Content-Type: text/csv; charset = ISO-8859-1; header = present

Strukturált szintaxis utótag

Az altípus nevének a '+' karaktert követő részét nevezik így. **A média típus szerkezetét jelzi.** (Például JSON vagy XML). Ezeket is regisztrálni kell. *Jelenleg használhatóak:* +ber, +cbor, +cbor-seq, +der, +fastinfoset, +gzip, +json, +json-seq, +jwt, +sqlite3, +tlv, +wbxml, +zip, +zstd, +xml, +yaml

Felső szintű típusok

application

A többi felső szintű típus alá nem sorolható adatokhoz, főleg alkalmazói programok által feldolgozott adatokhoz.

Média típus	Formátum
application/gzip	GZIP
application/javascript	ECMAScript
application/json	JSON

Média típus	Formátum
application/octet-stream	Tetszőleges bináris adatok
application/xhtml+xml	XHTML
application/xml	XML
application/zip	ZIP

audio

Média típus	Formátum
audio/flac	FLAC (Free Lossless Audió Codec)
audio/mp4	Videó adatokat nem tartalmazó MP4 állomány
audio/ogg	Túlnyomórészt hangokat tartalmazó Ogg állomány
audio/webm (nem regisztrált)	Videó adatokat nem tartalmazó WebM állomány

font

Média típus	Formátum
font/ttf	TrueType
font/woff	WOFF 1.0
font/woff2	WOFF 2.0

image

Média típusok	Formátum
image/jpeg	JPEG
image/png	PNG
image/svg+xml	SVG
image/webp	WebP

message, multipart

Összetett típusok, melyek lehetővé teszik különféle média típusú objektumok egységbe zárását.

Média típusok	Formátum
message/http	HTTP üzenet
multipart/byteranges	Byte-tartományok (részleges HTTP GET kérés eredményeként)
multipart/form-data	Űrlap adatok

model

Viselkedési vagy fizikai modell reprezentációik típusai (például 3D modellek)

Média típusok	Formátum
model/x3d-vrml	X3D
model/x3d+fastinfoset	X3D
model/x3d+xml	X3D

text

Sok idetartozó altípus definiál egy *charset* paramétert a karakterkódolás meghatározásához.

Média típusok	Formátum
text/css	CSS
text/csv	CSV
text/html	HTML
text/markdown	Markdown
text/plain	Sima szöveg
text/xml	Ugyanaz, mint az application/xml

video

Média típusok	Formátum
video/mp4	Videó adatokat (is) tartalmazó MP4 állomány
video/ogg	Vizuális megjelenítést igénylő tartalmat (is) magába foglaló Ogg állomány
video/webm (nem regisztrált)	Videó adatokat is tartalmazó WebM állomány

URI

Egységes erőforrás-azonosító (uniform resource identifier). Absztrakt vagy fizikai erőforrást azonosító tömör karaktersorozat. Egy erőforrás nem feltétlenül érhető el a Weben. URI-kat hozzá lehet rendelni akár a tárgyi világ objektumaihoz és fogalmakhoz is.

URI sémák

Minden URI egy **sémanévvel kezdődik**, melyet egy ':' karakter választ el a séma-specifikus résztől. A séma-specifikus rész szintaxisát és jelentését sémaspecifikációk határozhatják meg bizonyos korlátok között. Az URI-sémákat adminisztráló szerv: **IANA** (Internet Assigned Numbers Authority)

Közismert URI sémák:

- **file:** file:///usr/lib/R/library \
- **http/https:** https://url.spec.whatwg.org/#references \
- **mailto:** mailto:jeszenszky.peter@inf.unideb.hu \
- **about:** about:downloads

URI karakterek, százalékos kódolás

URI-kban megengedett karakterek:

- **Fenntartott karakterek:** Határolójelként használt karakterek ':', '/', '?', '#', '[', ']', '@', '!', '\$', '&', "'", '(', ')', '*', '+', '=', '&', ''', '', '$', '#', '!', ' ', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '', '
', '	', '', '', '', '', '', '', '', '', '�'
- **Nem fenntartott karakterek:**
 - 'A', ..., 'Z', 'a', ..., 'z'
 - '0', ..., '9'
 - '-', '.', '_', '~'

A specifikáció nem határoz meg karakterkódolást.

Százalékos kódolás (percent-encoding)

Nem megengedett karakterek használatához vagy fenntartott karakterek speciális jelentésének elnyomásához. Tekintsük a karaktert az adott karakterkódolásban ábrázoló **oktettssorozatot**. Az oktettssorozatot kódoljuk egy olyan karakterlánccal, melyben minden oktettet **%HH** módon ábrázolunk, ahol HH az oktett értékét reprezentáló két hexadecimális számjegy karakter.

- Szóközkarakter: %20
- Használhatóak hexadecimális számjegy karakterek is. Ebben az esetben nem számít, hogy kis- vagy nagybetűk szerepelnek.

file:///media/Movies/What's Up, Tiger Lily? (1966)/

Átalakítva: file:///media/Movies/What%26s%20Up%2C%20Tiger%20Lily%3F%20%281966%29/

URI szintaxis, a host, port, útvonal, lekérdezés és erőforrásrész-azonosító komponensek

URI szintaxis

Hierarchikus felépítése van. A komponensek felsorolása balról jobbra haladva fontosság szerint csökkenő sorrendben történik. \

Általános szintaxis:

séma ':' hierarchikus-rész ['?' lekérdezés] ['# erőforrásrész]

A **hierarchikus rész** egy autoritás (authority) és egy útvonal (path) komponenst tartalmazhat, szintaxisa: \ '/' autoritás útvonal vagy útvonal

Ha van autoritás komponens, akkor az útvonal üres kell, hogy legyen vagy a '/' karakterrel kell, hogy kezdődjön.

séma (scheme)	://	host	/	útvonal (path)	?	lekérdezés (query)	#	erőforrás-rész (fragment)
https	://	wordery.com	/	search	?	term=scotland	#	result

Ha nincs autoritás komponens, akkor az útvonal nem kezdődhet két '/' karakterrel.

séma (scheme)	:	útvonal (path)	?	lekérdezés (query)
mailto	:	jeszenszky.peter@inf.unideb.hu	?	subject=URI

Autoritás komponens

Nevét onnan kapta, hogy fennhatósága alá tartozik az URI további része által meghatározott névtér. \

Szintaxisa: [userinfo '@'] host [':' port]

Az URI sémák meghatároznak egy alapértelmezett portot. Például a http sémánál 80 az alapértelmezés.

Útvonal komponens

Útvonal részek '/' karakterekkel elválasztott sorozata, amely nem lehet üres.

Az első '?' vagy '#' karakterig, ezek hiányában pedig az URI végéig tart.

Az állományrendszerekben megszokott módon használhatóakk útvonal részként '.' és '..'.

Lekérdezés komponens

A '?' karakter jelzi az elejét, a '#' karakterig, annak hiányában pedig az URI végéig tart. Nem hierarchikus adatokat tartalmaz.

Gyakran név '=' érték formájú, '&' karakterekkel elválasztott név-érték párokat tartalmaz.

Erőforrásrész-azonosító

A '#' karakter jelzi az elejét, az URI végéig tart. Lehetővé teszi egy másodlagos erőforrás közvetett azonosítását egy elsődleges erőforrásra történő hivatkozáson keresztül. A másodlagos erőforrás lehet például az elsődleges erőforrás egy része.

Jelentését az elsődleges erőforrás elérése során kapott lehetséges reprezentációk határozzák meg, ezek média-típusa. A média típusok megszorításokat szabhatnak az erőforrásrész-azonosító formájára, meghatározhatják az így azonosított másodlagos erőforrások jelentését.

Hivatkozás-feloldás során mindig eltávolításra kerül.

A séma specifikációk olyan URI szintaxist kell, hogy meghatározzanak, melynek csak abszolút (erőforrásrész-azonosítót nem tartalmazó) URI-k felelnek meg.

Erőforrásrész-azonosító jelentése

- **text/html média típus:** Az erőforrásrész azonosító a dokumentum adott részét jelenti vagy állapot információt szolgáltat szkriptek számára. Az erőforrásrész-azonosító feldolgozását részletesen a HTML5 specifikáció határozza meg.
- **application/xml, text/xml és */*+xml média típusok:** Az erőforrásrész-azonosító szintaxisa és jelentése az XPointer Framework specifikáción alapul.

Hivatkozás

Abszolút URI

Olyan URI, amely nem tartalmaz erőforrásrész-azonosítót.

URI-hivatkozás

URI vagy relatív hivatkozás

```
http://www.gnu.org/licenses/licenses.html
/pub/linux/kernel/v3.x/testing/
../.. /images/bullet.png
index.html#contents
contacts.xml#element(/1/2)
#nav
gpl.html
<üres karakterlánc>
```

Same-document reference

Olyan URI hivatkozás, melynek feloldása az erőforrásrész-azonosítótól eltekintve a bázis URI-val azonos URI-t eredményez.

- Példáúé <üres karakterlánc>, #nav
- Hivatkozás-feloldás nem eredményezheti az erőforrás újbóli letöltését.

Bázis-URI meghatározása

Bizonyos média-típusoknál a relatív hivatkozások bázis URI-ja beágyazható a tartalomba. Így például dokumentumok definiálhatják a saját magukon belül érvényes bázis URI-t.

- XML: a bázis-URI-t az xml:base attribútum szolgáltatja
- HTML: a bázis-URI-t a :base attribútum szolgáltatja

Ha **nincs beágyazott bázis-URI** és a dokumentumot **egy másik entitás** - például egy másik dokumentum - **foglalja magában**, akkor a bázis URI ennek a bázis URI-ja.

Ha **nincs ilyen befoglaló entitás** sem, akkor a bázis-URI az az URI lesz, amelyen a dokumentumot elérték (átírányítás esetén az utolsó használt URI).

Egyébként a bázis-URI alkalmazásfüggő.

Relatív hivatkozás

Kb. egy URI séma-specifikus része, vagy annak egy megfelelő végszelete (lehet akár az üres karakterlánc is).

- A 'relatív URI' kifejezést a specifikáció egyáltalán nem használja!
- Míg egy URI mindig a használat környezetétől függetlenül azonosít egy erőforrást, egy relatív hivatkozás egy adott környezetben értelmezett.
- Egy úgynevezett bázis-URI alapján URI-vá lehet feloldani.
- A relatív hivatkozások feloldásához egy algoritmust ír le a specifikáció

Relatív hivatkozás feloldása

Legyen a bázis-URI: **http://example/a/b/c?q**

Relatív hivatkozás	Eredmény URI	Relatív hivatkozás	Eredmény URI
#z	http://example/a/b/c?q#z	d	http://example/a/b/d
"" (üres karakterlánc)	http://example/a/b/c?q	./d	http://example/a/b/d
.	http://example/a/b/	/d	http://example/d
./	http://example/a/b/	//localhost	http://localhost
..	http://example/a/	?y	http://example/a/b/c?y
../d	http://example/a/d	d?y	http://example/a/b/d?y
.././d	http://example/d		

HTML kódban lévő hivatkozás feloldás

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Example</title>
    <base href="http://example/docs/howto/">
    <link rel="stylesheet" type="text/css" href="theme.css">
  </head>
  <body>
    <a href="/about">
      
    </a>
  </body>
</html>
```

theme.css -> http://example/docs/howto/theme.css

/about -> http://example/about

../images/logo.png -> http://example/docs/images/logo.png

URI-k összehasonlítása

A séma és a host komponensek kisbetű-nagybetű érzéketlenek. A többi komponensnél kisbetű-nagybetű érzékenységet kell feltételezni, hacsak nem ír elő kisbetű-nagybetű érzéketlenséget a séma.

Tehát ekvivalensek például http://www.w3.org/ és HTTP://www.W3.org/ URI-k. **Ekvivalencia:**

- URI-k akkor ekvivalensek, ha ugyanazt az erőforrást azonosítják. (ez a definíció gyakorlati szempontból használhatatlan, mivel általában nincs mód az erőforrások összehasonlítására.)
- A gyakorlatban az ekvivalencia megállapítása az URI karakterláncok összehasonlításán alapul. Az összehasonlítás során normalizálás (nagybetű karakterek kisbetű karakterekké alakítása a kisbetű-

nagybetű érzéketlen komponensekben.)

Webböngészők

Az első böngésző: WorldWideWeb 1990. december 25. Tim Berners-Lee nevéhez kapcsolódik.

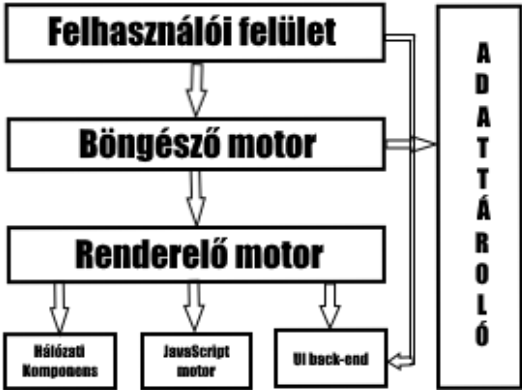
A böngészők felépítése és komponensei

A böngészőmotor, renderelő motor és layout engine kifejezéseket gyakran egymás megfelelőiként használják, bár ez technikailag pontatlan. A Gecko egy speciális eset, mivel egy böngészőmotorból és egy renderelő motorból áll.

Felhasználói felület

A felhasználói felülethez tartozik a címsor, a gombok (például, vissza, előre, frissítés), menük, stb.

Beletartozik a böngésző minden látható része, annak a területnek a kivételével, ahol a weboldal megjelenítésre kerül.



Böngészőmotor

Egy magas szintű interfészt biztosít a renderelő motor manipulálásához. Közvetítő szerepet tölt be a felhasználói felület és a renderelő motor között. Kezeli az adattároló komponenst.

Renderelő motor

A böngészők egy kulcskomponense.

Feladata webes tartalom (például egy HTML vagy XML dokumentum) vizuális reprezentációjának létrehozása egy kimeneti eszközön való megjelenítés céljából.

A tartalom renderelése a meghatározott formázási szabályoknak megfelelően történik. A böngészők a renderelő motor egy külön példányát futtathatják minden fülhöz.

Hálózati komponens

A hálózati komponens felelős a hálózati kommunikáció kezeléséért, beleértve: HTTP kérések és válaszok, WebSocket API, WebRTC.

Egy platformfüggetlen interfészt biztosít, mely platform specifikus API-k kerülnek felhasználásra.

JavaScript-motor

Feladata JavaScript kód végrehajtása. A JavaScript kód először bájtkódra lesz lefordítva. Ezután a bájtkód futásidejű fordítással (just-in-time (JIT) compilation) kerül lefordításra gépi kódra.

Fő JavaScript-motorok

Név	Programozási nyelv	Licenc	Minek a motorja?
SpiderMonkey	C++/Rust/JavaScript	Mozilla Public License 2.0	Mozilla

Név	Programozási nyelv	Licenc	Minek a motorja?
V8	C++	New BSD License	Chromium
JavaScriptCore	C++	GPLv2	Apple

UI back-end

Az UI back-end felelős a böngészőben a rajzolásért. Egyaránt használják a felhasználói felület elemek és a weboldal megjelenítéséhez. Egy platformfüggetlen interfészt biztosít, mely mögött platform specifikus API-k kerülnek felhasználásra. A teljesítmény növeléséhez a GPU-t használhatja.

Adattároló

Az adattároló felelős az adatok perzisztens tárolásáért, a következőkhöz használják: HTTP sütik, HTTP gyorsítótárazás, Indexed Database API (IndexedDB), Web Storage API

Weboldalak renderelésének lépései, kritikus renderelési útvon

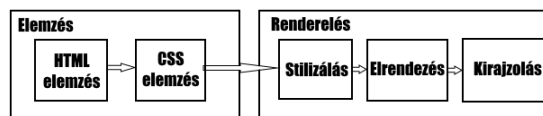
Weboldalak renderelésének lépései

- **Navigálás:** a felhasználó beírja az oldal címét a címsorba
- **IP-cím meghatározás:** Egy DNS-lekérdezés történik a szerver IP-címének meghatározásához. (További lekérdezések is kellhetnek)
- **Hálózati kapcsolat:** Egy hálózati kapcsolat (például TCP) kapcsolat jön létre a szerverrel.
- **HTTP kérés:** A böngésző egy első HTTP kérést küld, melyre a szerver egy HTTP választ küld vissza, mely az esetek többségében egy HTML reprezentációt tartalmaz.
- **Kritikus renderelési útvon:** A folyamat a kritikus renderelési útvonallal folytatódik.
- **További feladatok:** JavaScript fordítás, akadálymentességi (accessibility) fa létrehozása.

Kritikus renderelési útvon

A kritikus renderelési útvon (critical rendering path) azoknak a lépéseknek a sorozata, melyeket a böngésző végrehajt egy HTML vagy XML dokumentum egy kimeneti eszközön történő renderelésekor.

1. **HTML elemzés:** DOM fa építése
2. **CSS elemzés:** CSSOM fa építése
3. **Stilizálás:** dobozfa (render tree, box tree) építése. Nem kerülnek a fába azok az elemek, melyek nem kerülnek megjelenítésre, mint például a *title* vagy a *script*.
4. **Elrendezés:** méret és pozíció meghatározása a dobozfa csomópontjaihoz
 1. A *layout* kifejezést használjuk, amikor először kerül meghatározásra egy csomópont mérete és pozíciója
 2. A méret és pozíció későbbi újraszámítása: *reflow*
 3. A még ismeretlen méretű helyettesített elemek (például képek) számára kezdetben pontosan nem meghatározott (*placeholder*) hely kerül lefoglalásra. A pontos méret meghatározásakor egy *reflow* történik.
5. **Kirajzolás:** a dobozfa csomópontjainak kirajzolása a kimeneti eszközön. A teljesítmény a GPU bevonásával javítható.



A fő renderelő motorok, a fő asztali és mobil böngészők

A fő renderelő motorok

	WebKit	Blink	Gecko
Fejlesztő	Apple	Chromium Project (Google)	Mozilla Project
Licenc	LGPLv2 / Simplified VSD License	BSD License / New BSD License / LGPLv2.1	Mozilla Public License 2.0
Programozási nyelv	C++	C++	C, C++, JavaScript, Rust
Szoftverek	Safari, Google Chrome for iOS, Firefox for iOS, GNOME Web	Chromium, Google Chrome, Google Chrome for Android, Chromium-based Microsoft Edge, Opera, Vivaldi	Firefox, Firefox for Android, SeaMonkey

A fő asztali és mobil böngészők

A fő asztali böngészők

Név	Fejlesztő	Licenc	Programozási nyelv	Platform	Renderelő motor	JavaScript motor	Technikai információk
Chromium	Chromium Project (Google)	New BSD License	C, C++, JavaScript	Linux, macOS, Windows	Blink	V8	chrome://system
Google Chrome	Google	nem szabad	C, C++, JavaScript	Linux, macOS, Windows	Blink	V8	chrome://system
Firefox	Mozilla Project	Mozilla Public License 2.0	C, C++, JavaScript, Rust	Linux, macOS, Windows	Gecko	SpiderMonkey	about:support
Opera	Opera Software	nem szabad	C++	Linux, macOS, Windows	Blink	V8	opera://system
Safari	Apple	nem szabad	C++, Objective-C	macOS	WebKit	JavaScriptCore	
Microsoft Edge	Microsoft	nem szabad		Linux (2020-), macOS (2019-), Windows	EdgeHTML (-2019), Blink (2019-)	Chakra (-2019), V8 (2019-)	

A fő mobil böngészők

Név	Fejlesztő	Licenc	Platform	Renderelő motor	JavaScript motor
Google Chrome (Android, iOS)	Google	nem szabad	Android, iOS	Blink (Android), iOS WebKit (iOS)	V8 (Android), JavaScriptCore (iOS)
Safari for iOS	Apple	nem szabad	iOS	iOS WebKit	JavaScriptCore

Név	Fejlesztő	Licenc	Platform	Renderelő motor	JavaScript motor
Samsung Internet for Android	Samsung	nem szabad	Android	Blink	V8
UC Browser (Android, iOS)	UCWeb, az Alibaba egy leányvállalata	nem szabad	Android, iOS	Blink (Android), iOS WebKit (iOS)	V8 (Android), JavaScriptCore (iOS)

Chromium és Google Chrome

A Google Chrome a Chromiumon alapul. **A két böngésző közötti különbségek:**

- Eltérő logóik vannak.
- Eltérő audio és videó kodekekkel vannak felszerelve.
- A felhasználói adatok tárolása különböző könyvtárakban.
- A Chromium forráskódját módosíthatják az operációs rendszer disztribúciók, a csomagolás is disztribúciófüggő.

Böngésző kiegészítők, böngésző kiterjesztések

Egy **böngésző kiegészítő** egy böngésző megjelenését vagy viselkedését testreszabó szoftvermodul. A kiegészítő *fajtái*: kiterjesztések, nyelvi csomagok, témák, plugin-ek

Egy **böngésző kiterjesztés** egy olyan kiegészítő, mely egy böngésző viselkedését módosítja, azaz új funkcionalitást ad hozzá vagy létező funkcionalitást módosít.

"Fej nélküli" (headless) böngészők

Egy "fej nélküli" (headless) böngésző egy grafikus felhasználói felület nélküli böngésző. Programozottan vezérelhető.

Lehetséges alkalmazások:

- Webalkalmazások automatikus tesztelése
- Weboldakkal történő interakció automatizálása
- Képernyőképek készítése weboldalakról
- Információ kinyeré weboldalakból (web scraping)

A Chromium, Google Chrome és Firefox böngészők futtathatnak "fej nélküli" módban

Az about URI séma

Az about URI sémát széles körben használják a böngészők ahhoz, hogy hozzáférést adjanak a belső forrásaikhoz, mint például a beállítások, alkalmazás információk vagy rejtett beépített funkciók. Például az about:black URI egy üres oldalra hivatkozik.

Bizonyos böngészők az about URI-kat a saját ekvivalens megfelelőikre képezik le, a sémanevet (about) a nevükre (mint például chrome, opera vagy edge) kicserélve.

HTTP alapok

A http és https URI-séma

A HTTP definiálja a http és https URI sémákat. Egy http vagy https URI **eredet szervert** a hoszt azonosító és az opcionális portszám határozza meg. Az útvonal komponens és az opcionális lekérdezés komponens egy lehetséges **cél erőforrást** azonosít az eredet szervert névterében.

Egy http vagy https URI előfordulása nem jelenti azt, hogy mindig egy HTTP szervert figyel az URI által meghatározott eredet szerverten.

A https sémán keresztül elérhető erőforrásoknak nincs közös identitása a http sémával.

http URI séma

Célja, hogy lehetővé tegye erőforrások azonosítását egy olyan potenciális eredet szerverten, mely egy adott ponton vár TCP kapcsolatokra.

URI szintaxis:

"http://" host [":" port] [útvonal] ["?" lekérdezés]

Ha nincs megadva vagy üres a port alkomponens, akkor a 80-as TCP port az alapértelmezés.

https URI séma

Célja, hogy lehetővé tegye erőforrások azonosítását egy olyan potenciális eredet szerverten, mely egy adott porton vár TCP kapcsolatokra és mely képes HTTP kommunikációhoz biztonságossá tett TL kapcsolat létrehozására.

URI szintaxis:

"https://" host [":" port] [útvonal] ["?" lekérdezés] Ha nincs megadva vagy üres a port alkomponens, akkor a 443-as TCP port az alapértelmezés.

http és https URI összehasonlítása

Az üres útvonal ekvivalens a "/" útvonallal. A séma és a host komponensek kisbetű-nagybetű érzéketlenek és megadásuk rendszerint kisbetűkkel történik. A többi komponens összehasonlítása kisbetű-nagybetű érzékeny módon történik. A nem fenntartott karakterek ekvivalensek a százalékos kódolással megadott formájukkal. Ekvivalens HTTP URI-król feltételezhető, hogy ugyanazt az erőforrást azonosítják.

A HTTP minden egyes főverziója saját szintaxist határoz meg az üzenetváltáshoz, melyet keretezési mechanizmusnak (framing mechanism) is neveznek.

Üzenet absztrakció

Az RFC 9110 az üzenetek egy olyan absztrakcióját határozza meg, mely szerint egy üzenet a következőkből áll:

Vezérlő adatok

Az üzenetek az elsődleges céljukat leíró vezérlő adatokkal kezdődnek.

- Kérésekben a vezérlő adatok a metódust, a kérést célt és a protokoll verzióját foglalják magukban.
- Válaszokban a vezérlő adatok az állapotkódot, az opcionális indok frázist és a protokoll verziót foglalják magukban.

Minden HTTP üzenetnek van protokoll verziója.

Fejléc szakasz

A tartalom előtt küldött vagy fogadott mezőket fejlécmezőknek (vagy informálisan egyszerűen fejlécnek) nevezzük. Egy üzenet fejléc szakasza fejléc mezősorok egy sorozatából áll.

Tartalom

A HTTP üzenetek egy teljes vagy részleges reprezentációt hordozhatnak az üzenet tartalmaként. A tartalom átvitele egy bájtfolyamként történik a fejléc szakasz után. A Content-Type és a Content-Encoding fejlécmezők által meghatározott formátumban és kódolásban van.

Tartalom szemantika

A tartalom célját egy kérésben a metódus szemantika határozza meg.

- Például egy reprezentáció egy POST kérés tartalmaként a cél erőforrás által feldolgozandó információkat ábrázol.

Egy válaszban a tartalom célját a kérés metódusa, az állapotkód és a tartalmat leíró válasz mezők határozzák meg.

- Például egy GET kérésre adott 200 (OK) állapotkódú válasz a cél erőforrás az üzenet létrehozásának pillanatában megfigyelhető aktuális állapotát ábrázolja.

Lezáró szakasz

A tartalom után küldött vagy fogadott mezőket **lezáró mezőknek** (vagy informálisan egyszerűen lezárónak) nevezzük. A lezáró mezők ellenőrző összegek, digitális aláírások, kézbesítési metrikák vagy utófeldolgozási állapot információk továbbítására használhatók. Egy üzenet lezáró szakasza lezáró mezősorok egy sorozatából áll. A lezáró mezőket a fejlécmezőktől elkülönítve ajánlott feldolgozni és tárolni.

Ez az üzenet absztrakció egy több HTTP verziót átfogó általánosítás, melynek vannak olyan lehetőségei, melyek bizonyos verziókban nem találhatók meg. Az üzemeteket önleírónak szánják, azaz minden, amit egy fogadónak tudnia kell az üzenetről, megállapítható a (dekódolt) üzenet vizsgálatával.

Mezők

A HTTP mezőket használ adatok név/érték párok formában történő szolgáltatásához.

Információk továbbítása:

Mezők szolgálnak a következő információk továbbítására:

- magát az üzenetet leíró metaadatok kérésekben és válaszokban (például Date)
- reprezentáció metaadatok kérésekben és válaszokban (például Content-Type)
- információk a kliensről kérésekben (például User-Agent)

- információk a szerverről válaszokban (például Server)
- erőforrás metaadatok válaszokban (például Last-Modified)

Mezők küldése és fogadása

A **mezők küldése és fogadása** az üzenetek fejléc és lezáró szakaszaiban történik.

- Egy üzenet fejléc vagy elzáró szakaszában küldött mezőt fejlécmezőnek illetve lezáró mezőnek nevezünk.
- Bizonyos mezők, mint például az ETag, előfordulhatnak fejléc- vagy lezáró mezőként is.

Mezőnevek

Egy mezőnév egy vagy több karakterből álló olyan sorozat, melyben az US-ASCII karakterkészlet egy részhalmaza használható csak. A mezőnevek kisbetű-nagybetű érzéketlenek.

Mezőértékek

Egy mezőérték egy olyan karaktersorozat, mely egy vagy több nyomtatható US-ASCII karakterből, szóközből és vízszintes tabulátorból áll. A vezető és záró whitespace karaktereket a felhasználás előtt el kell távolítani. A mezőkhöz előírható, hogy egyetlen tagot vagy pedig egy vesszővel elválasztott taglistát hordozzanak. Mind egyes mező korlátozhatja a megengedett értékek halmazát.

Mezőszakaszok

Tetszőleges számú olyan mezősorból állnak, melyek mindegyike egy mezőnevet és egy hozzá tartozó mezősor értéket tartalmaz. Amikor egy mezőnév megismétlődik egy szakaszban, az érték egy olyan lista, melyben a mező a szakaszbeli mezősor értékei az előfordulásuk sorrendjében kerülnek összefűzésre elválasztóként egy vessző karakterrel.

- A gyakorlatban a Set-Cookie fejlécmező gyakran több mezősorban is megjelenik egy válaszban, azonban a Set-Cookie mezősor értékek nem kerülnek egyesítésre egyetlen mezőértékké.

Nem lényeges, hogy milyen sorrendben fordulnak elő különböző mezőnevű mezősorok egy szakaszban.

További információk

A HTTP specifikációk sok szabványos mezőt határoznak meg. A HTTP által meghatározottakon túl sok más specifikáció definiál fejlécmezőket. Eltérő rendelkezés hiányában a mezők az összes HTTP verzióhoz kerülnek meghatározásra. Az IANA adminisztrálja a HTTP mezőket.

A User-Agent fejlécmező

A felhasználói ágensről tartalmaz információkat, ahonnan a kérés származik. Felhasználható a válasz testreszabásához vagy pedig böngésző vagy operációs rendszer használatra vonatkozó elemzésekre. A felhasználói ágens számára ajánlott minden egyes kérésben elküldeni a fejlécmezőt. A mezőérték egy vagy több termékazonosítóból áll, melyek mindegyikét nulla vagy több megjegyzés követi.

- A termékazonosítók felsorolása a fontosságuk szerinti csökkenő sorrendben történik.
- Minden egyes termékazonosító egy névből és egy opcionális verziószámból áll.
- A megjegyzéseket zárójelek határolják.

```
curl: curl/8.8.0  
Firefox: Mozilla/5.0 (X11; Linux x86_64; rv:130.0) Gecko/20100101 Firefox/130.0
```

HTTP metódusok

Elsődlegesen a metódusok jelzik a kérések célját. A metódus szemantika tovább specializálható a kérésben bizonyos fejlécmezőkkel (lásd például a feltételes kérés fejlécmezőket). A metódusnevek kisbetű-nagybetű érzékenyek. A specifikáció által meghatározott szabványos metódusok:

- CONNECT
- DELETE
- GET
- HEAD
- OPTIONS
- POST
- PUT
- TRACE

Minden általános célú szervnek támogatnia kell a GET és a HEAD metódusokat, az összes többi metódus opcionális. További HTTP metódusok is meghatározásra kerültek. Az IANA adminisztrálja a HTTP metódusokat.

Egy cél erőforrás által megengedett metódusok az Alloe fejlécmezőben sorolhatók fel. Ha egy kérés metódusát nem ismeri fel vagy nem implementálja egy eredet szerver, akkor az 501 (Not Implemented) állapotkóddal ajánlott válaszolnia. Ha egy kérés metódusát felismeri és implementálja ugyan egy eredet szerver, de nem engedélyezi a cél erőforrásra, akkor a 405 (Method Not Allowed) állapotkóddal ajánlott válaszolnia.

GET

A cél erőforrás egy aktuális kiválasztott reprezentációjának átvitelét kéri. Az információ-visszakérés elsődleges mechanizmusa. Egy kliens úgy módosíthatja a GET jelentését a kérésben a Range fejlécmező küldésével, hogy az csak a kiválasztott reprezentáció bizonyos részeinek átvitelét kéri.

HEAD

Azonos a GET metódussal, azzal a különbséggel, hogy a szerver nem küldhet tartalmat a válaszban. Úgy használható metaadatok szerzésére a kiválasztott reprezentációról, hogy reprezentáció adatok nem kerülnek átvitelre.

```
$ curl --head http://www.r-project.org/  
HTTP/1.1 200 OK  
Date: Sun, 15 Sep 2024 12:34:32 GMT  
Server: Apache  
Last-Modified: Fri, 14 Jun 2024 08:10:02 GMT  
ETag: "1b27-61ad5247107c3"  
Accept-Ranges: bytes  
Content-Length: 6951  
Vary: Accept-Encoding  
Content-Type: text/html
```

POST

Azt kéri, hogy a cél erőforrás dolgozza fel a kérésben mellékelt reprezentációt a saját szemantikájának megfelelően. **Tipikus felhasználások:**

- Adatok (például űrlap adatok) küldése egy adatfeldolgozó folyamat számára.
- Egy üzenet postázása egy hírcsoportba, levelezési listára vagy blogba.
- Egy új erőforrás létrehozása
- Adatok összefűzése egy erőforrás létező reprezentációjához.

```
$ http --form https://httpbin.org/post number=42 text="Hello World!" -v POST /post
HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
Content-Length: 32
Content-Type: application/x-www-form-urlencoded; charset=utf-8
Host: httpbin.org
User-Agent: HTTPie/3.2.3

number=42&text=Hello%2C+World%21
```

```
$ http --form --multipart https://httpbin.org/post \ number=42 text="Hello World!" -
v
POST /post HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate, br, zstd
Connection: keep-alive
Content-Length: 225
Content-Type: multipart/form-data; boundary=f067108d371949f2a8c4fdbb991afac1
Host: httpbin.org
User-Agent: HTTPie/3.2.3

--90f9bc6b7be04d918e70eda8ef004ec8
Content-Disposition: form-data; name="number"

42
--90f9bc6b7be04d918e70eda8ef004ec8
Content-Disposition: form-data; name="text"

Hello, World!
--90f9bc6b7be04d918e70eda8ef004ec8-
```

ide van még egy példa.....

PUT

Azt kéri, hogy kerüljön létrehozásra vagy helyettesítésre a cél erőforrás állapota a kérésben mellékelt reprezentáció által meghatározott állapottal. Egy adott reprezentációt tartalmazó sikeres PUT kérés arra enges

következtetni, hogy egy következő GET kérés ugyanarra a cél erőforrásra egy 200 (OK) állapotkódú válaszban elküldött ekvivalens reprezentációt eredményez.

```
$ echo "Hello, World!" | http PUT https://transfer.sh/hello.txt \  
    Content-Type:text/plain-v  
PUT /hello.txt HTTP/1.1  
Accept: application/json, */*;q=0.5  
Accept-Encoding: gzip, deflate  
Connection: keep-alive  
Content-Length: 14  
Content-Type: text/plain  
Host: transfer.sh  
User-Agent: HTTPie/3.2.2  
  
Hello, World!  
  
HTTP/1.1 200 OK  
Content-Length: 40  
Content-Type: text/plain  
Date: Mon, 07 Aug 2023 10:52:58 GMT  
Server: Transfer.sh HTTP Server  
X-Made-With: <3 by DutchCoders  
X-Served-By: Proudly served by DutchCoders  
X-Url-Delete:  
  
https://transfer.sh/fUMsV053HK/hello.txt/SYNHIEOxWNFhDsEhKpf7  
https://transfer.sh/fUMsV053HK/hello.txt
```

DELETE

Azt kérelmezi, hogy az eredet szerver törölje a cél erőforrás és aktuális funkcionalitása közötti kapcsolatot. Ha a cél erőforrásnak egy vagy több aktuális reprezentációja van, akkor ezeket az eredet szerver vagy megsemmisíti, vagy nem, a kapcsolódó tárterület vagy felszabadításra kerül, vagy nem, teljes egészében az erőforrás természetétől és az eredet szerver általi implementációtól függően. Viszonylag kevés erőforrás engedi meg a DELETE metódust.

```
$ http DELETE https://transfer.sh/fUMsV053HK/hello.txt/SYNHIEOxWNFhDsEhKpf7-v  
DELETE /fUMsV053HK/hello.txt/SYNHIEOxWNFhDsEhKpf7 HTTP/1.1  
Accept: */*  
Accept-Encoding: gzip, deflate  
Connection: keep-alive  
Content-Length: 0  
Host: transfer.sh  
User-Agent: HTTPie/3.2.2  
  
HTTP/1.1 200 OK  
Content-Length: 0  
Date: Mon, 07 Aug 2023 10:57:30 GMT  
Server: Transfer.sh HTTP Server  
X-Made-With: <3 by DutchCoders  
X-Served-By: Proudly served by DutchCoders
```

Állapotkódok

Egy válasz állapotkódja egy olyan háromjegyű decimális egész szám, mely a kérés eredményét és a válasz jelentését írja le, beleértve azt, hogy sikeres volt-e a kérés és hogy milyen tartalom van benne mellékelve (ha van egyáltalán). Az összes érvényes állapotkód a 100 és 599 közötti tartományba esik.

Állapotkódok fajtái

Az állapotkód első számjegye a válasz fajtáját határozza meg:

- 1xx (informáló): a kapcsolat állapotát vagy a kérés előrehaladását közlő köztes választ jelez egy végső válasz küldése előtt.
- 2xx (siker): azt jelzi, hogy a kérés sikeresen fogadásra, értelmezésre és elfogadásra került.
- 3xx (átírányítás): azt jelzi, hogy a felhasználói ágens további műveletet kell, hogy végrehajtson a kérés teljesítéséhez, melyet automatikusan elvégezhet.
- 4xx (kliens hiba): azt jelzi, hogy a kérés rossz szintaxisú vagy nem teljesíthető.
- 5xx (szerver hiba): azt jelzi, hogy a szerver nem teljesített egy nyilvánvalóan érvényes kérést.

Állapotkódok

Egy kliensnek nem kell megértenie minden regisztrált állapotkód jelentését. Kötelező azonban az állapotkód fajtájának megértése az első számjegy alapján. Egy nem felismert állapotkódot az x00 állapotkóddal ekvivalensként kell kezelni, ahol x a nem felismert állapotkód első számjegye.

HEAD kérésekre adott válaszok kivételével a 4xx vagy 5xx állapotkódú válaszokban a szerver számára ajánlott egy olyan reprezentáció küldése, mely a hiba magyarázatát tartalmazza, valamint azt, hogy a hibát okozó körülmények átmenetiek vagy állandóak. Az állapotkódok kiterjeszthetők Az IANA adminisztrálja az állapotkódokat.

Főbb állapotkódok

Állapotkód	Indok	Leírás
100	Continue	A szerver megkapta a kérés bevezető részét és még nem utasította el. A szerver azután kíván egy végső választ küldeni, ha a teljes kérést megkapta és megfelelően járt el.
101	Switching Protocols	A szerver kész áll a kapcsolaton használt alkalmazási protokoll lecserélésére.
200	OK	A kérés sikeres volt. A válaszban küldött tartalom a metódustól függ. Például egy GET kérésre adott válasz tartalma a cél erőforrás egy reprezentációja.
201	Created	A kérés teljesítésre került, eredményül egy vagy több új erőforrás került létrehozásra.
202	Accepted	A kérés elfogadásra került, de a feldolgozása nem fejeződött be.
204	No Content	A szerver sikeresen teljesített egy tartományra vonatkozó kérést a kiválasztott reprezentáció egy vagy több részének átvitelével.
206	Partial Content	A szerver sikeresen teljesített egy tartományra vonatkozó kérést a kiválasztott reprezentáció egy vagy több részének átvitelével.
300	Multiple Choices	A cél erőforrásnak egynél több reprezentációja van. HEAD kérés kivételével ajánlott a válaszban tartalomként egy reprezentáció metaadatokból és URI

Állapotkód	Indok	Leírás
		hivatkozásokból álló lista elhelyezése, melyből a felhasználói ágens kiválaszthatja a számára legmegfelelőbbet.
301	Moved Permanently	A cél erőforrás URI-ja véglegesen megváltozott, a rá való további hivatkozásokhoz a mellékelt URI-k egyikét kellene használni.
302	Found	A cél erőforrás ideiglenesen egy másik URI alatt található.
303	See Other	A szerver a felhasználói ágenst egy másik erőforrásra irányítja át, melynek célja, hogy egy közvetett választ adjon az eredeti kérésre.
304	Not Modified	A szerver egy feltételes GET vagy HEAD kérést kapott. Nincs szükség arra, hogy elküldje a cél erőforrás reprezentációját, mivel a kérés azt jelzi, hogy a kliens érvényes reprezentációval rendelkezik.
400	Bad Request	A szerver nem tudja vagy nem fogja feldolgozni a kérést valamilyen kliens hiba miatt (például a kérés szintaktikailag hibás).
401	Unauthorized	A kérés nem tartalmaz érvényes hitelesítő adatokat a cél erőforráshoz.
403	Forbidden	A szerver ugyan megértette a kérést, de megtagadja a teljesítését. Ha a kérésben hitelesítő adatok vannak, akkor azokat nem megfelelőnek tekinti a szerver a hozzáféréshez.
404	Not Found	Az eredet szervernek nincs a cél erőforráshoz aktuális reprezentációja vagy nem kívánja azt nyilvánosságra hozni.
405	Method Not Allowed	Az eredet szerver ismeri ugyan a metódust, de a cél erőforrás nem támogatja azt.
500	Internal Server Error	Olyan váratlan körülmény lépett fel a szerveren, mely megakadályozza a kérés teljesítését.
501	Not Implemented	A szerver nem támogatja a kérés teljesítéséhez szükséges funkcionalitást (például metódust).
503	Service Unavailable	A szerver jelenleg nem képes a kérés kezelésére például ideiglenes túlterhelés vagy menetrend szerinti karbantartás miatt.

Átírányítás, a Location fejlécmező

```
$ curl--http1.1-v http://w3.org/
> GET / HTTP/1.1
> Host: w3.org
> User-Agent: curl/8.8.0
> Accept: */*
>
< HTTP/1.1 301 Moved Permanently
< Location: https://www.w3.org/
< ...
<
```

```
$ curl--http1.1-v-L http://w3.org/
> Host: w3.org
> User-Agent: curl/8.8.0
> Accept: */*
>
```

```
< HTTP/1.1 301 Moved Permanently
< Location: https://www.w3.org/
< ...
<
> GET / HTTP/1.1
> Host: www.w3.org
> User-Agent: curl/8.8.0
> Accept: */*
>
< HTTP/1.1 200 OK < ...
```

```
$ http https://dbpedia.org/resource/Hungary
HTTP/1.1 303 See Other
Connection: keep-alive
Content-Length: 0
Content-Type: text/html; charset=UTF-8
Date: Sun, 15 Sep 2024 12:55:33 GMT
Location: http://dbpedia.org/page/Hungary
Server: Virtuoso/08.03.3331 (Linux) x86_64-generic-linux-g
...
```

Tartalomjegyzetés, proaktív egyeztetés

Amikor a válaszok (akár sikert, akár hibát jelző) tartalmat hordoznak, az eredet szerver gyakran többféle módon is ábrázolhatja a tartalmat, például különböző formátumokban, nyelveken vagy kódolásokkal. A különböző felhasználóknak vagy felhasználói ágenseknek ugyancsak eltérő képességei, jellemzői vagy preferenciái lehetnek, melyek befolyásolhatják, hogy a rendelkezésre álló reprezentációk közül melyik lenne számukra a legjobb. Ezért biztosít a HTTP a tartalomjegyzetéshez mechanizmusokat. **Két főbb tartalomjegyzetési mintát határoz meg:**

1. **Proaktív egyeztetés (proactive negotiation):** a szerver választja ki a reprezentációt a felhasználói ágens kifejezett preferenciái alapján.
 1. **Szerver-vezérelt (server-driven)** tartalomjegyzetés néven is ismert.
2. **Reaktív egyeztetés (reactive negotiation):** a szerver választásra kínálja fel a felhasználói ágensnek a reprezentációk listáját
 1. **Ágens-vezérelt (agent-driven)** tartalomjegyzetés néven is ismert

Más tartalomjegyzetési minták is lehetségesek. Ezek a minták egymást nem kölcsönösen kizáróak.

Proaktív egyeztetés

Az előnyben részesített reprezentációt egy algoritmussal választja ki az eredet szerver a felhasználói ágens preferenciái alapján. A választás a válaszhoz rendelkezésre álló reprezentációk és a kérésben szolgáltatott különféle információk– beleértve az explicit tartalomjegyzetési fejlécmezőket, a User-Agent mező részeit és az olyan implicit jellemzőket, mint például a kliens hálózati címeösszehasonlítása alapján történik. Proaktív egyeztetésnek alávetett válaszban gyakran kerül küldésre a Vary fejlécmező annak jelzésére, hogy a kérés mely részei kerültek felhasználásra a kiválasztási algoritmusban.

Előnyök	Hátrányok
Amikor nehéz egy felhasználói ágens számára leírni a rendelkezésre álló reprezentációk közüli választás algoritmusát, vagy	Lehetetlen a szerver számára annak pontos meghatározása, hogy mi lenne a legjobb egy tetszőleges felhasználónak, mivel ehhez kimerítő ismeretek lennének szükségesek a felhasználói ágens képességeiről és a válasz tervezett felhasználási módjáról.
amikor a szerver az első válaszban el kívánja küldeni a felhasználói ágensnek az általa legjobbnak becsült reprezentációt, elkerülendő egy további kérés	Nagyon nem hatékony a felhasználói ágens képességeit minden kérdésben leírni, ez egyben a felhasználó magánszférájára is egy lehetséges kockázatot jelenthet.
	Bonyolítja az eredet szerver megvalósítását és a kérésekre válaszokat előállító algoritmusokat.
	Korlátozza a válaszok újrafelhasználhatóságát megosztott gyorsítótárazáshoz.

Az Accept fejlécmező

Értéke média tartományok egy vesszővel elválasztott listája, melyben minden egyes média tartományt 0 vagy több médiatípus paraméter (például charset) valamint egy opcionális relatív súly (azaz q paraméter) követhet.

Média tartomány:

- /: az összes médiatípust jelenti
- type/: az adott típus összes altípusát jelenti (például text/)
- type/subtype: az adott médiatípust jelenti (például text/html)

A mező értéke változhat, például eltérő lehet egy, a böngésző címsorában hivatkozott dokumentum vagy egy \$\$ HTML elembe hivatkozott kép letöltésekor.

Alapértelmezett érték a Firefox 92 számú vagy későbbi verzióiban:

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,/*;q=0.8

Média tartomány	q
text/html	1
application/xhtml+xml	1
image/avif	1
image/webp	1
application/xml	0.9
* / *	0.8

HTTP/1.1 üzenetformátum

Az üzenetek egy kezdősorral (start-line) kezdődnek, melyet CRLF követ. A kezdősort nulla vagy több fejlécsor kövei, melyeket együttesen fejléceknek vagy fejléc szakasznak neveznek. Egy üres sor jelzi a fejléc szakasz végét. Opcionálisan szerepelhet az üzenet végén egy üzenettörzs (message body).

Kérések kezdősorának szintaxisa

<metódus> <kérés-cél> "HTTP/1.1"

- A kérés-cél a cél erőforrást azonosítja, melyre a kérés vonatkozik.
 - Leggyakoribb formája: útvonal ["?" lekérdezés]
 - Ha a cél URI útvonal komponense üres, akkor a kliens "/"-t kell, hogy küldjön útvonalként.
 - A cél URI host és port komponense a Host fejlécmezőben kerül elküldésre.

Válaszok első sora

A válaszok első sorát állapotsornak nevezik és a következő szintaxisa van: "HTTP/1.1" <állapotkód> [indok_frázis]

- Az opcionális indok frázis az állapotkód egy szöveges leírása.
- Egy kliens számára ajánlott az indok fázis figyelmen kívül hagyása.

Mezősorok

Minden mezősor egy kisbetű-nagybetű érzéketlen mezőnévvel kezdődik, melyet egy pontosvessző opcionális vezető whitespace, egy mezősor érték és záró whitespace követ. A mezősorokat egy CRLF zárja.

Üzenettörzs

Egy HTTP/1.1 üzenet törzse (ha van) szolgál a kérés vagy válasz tartalmának hordozására. Az üzenettörzs azonos a tartalommal, feltéve hogy nem kerül alkalmazásra egy átviteli kódolás (például chunked).

Markdown

Jelölőnyelvek, példák jelölő nyelvekre

A jelölőnyelvek szöveg annotálására szolgáló számítógépes nyelvek. Lehetővé teszik szövegrészekhez metaadatok megadását a szövegtől jól elkülöníthető módon. **Példák jelölő nyelvekre**

- AsciiDoc: <https://asciidoc.org/>
- Markdown: <https://daringfireball.net/projects/markdown/>
- TeX, LaTeX: <https://www.latex-project.org/>
- troff (man oldalak): <https://gnu.org/software/groff/>
- Wikitext: <https://en.wikipedia.org/wiki/Help:Wikitext>
- XML: <https://www.w3.org/XML/>

Mi a Markdown?

A Markdown egy pehelysúlyú jelölőnyelv sima szöveges formázási szintaxissal.

A Markdown jellemzői

Könnyen olvasható és könnyen írható sima szöveges formátum. Különféle kimeneti formátumokba alakítható (például HTML-be). Kifejezetten a speciális ismeretekkel nem rendelkező felhasználókat célozza meg. Szintaxisát főleg a sima szöveges email formázás ihlette.

Szabványosítás, Markdown változatok

Szabványosítás

CommonMark <https://commonmark.org/> A Markdown egy egyértelmű szintaxis specifikációja. Egy tesztkészletet is biztosít az implementációk, specifikációnak való megfelelésének ellenőrzéséhez.

Markdown változatok

GitHub Flavored Markdown (GFM) Pandoc's Markdown

Gyakorlati felhasználások

A Markdown-t széles körben használják a weben szövegbevitelre

Főbb alkalmazási területek	Alkalmazások
Kollaborációs eszközök és platformok	GitHub, Microsoft Teams, Trello
Blogolási platformok és tartalomkezelő rendszerek	Moodle, WordPress
Online közösségi platformok	Discord, Reddit, Stack Overflox
Jegyzetelő alkalmazások	Joplin, memos
Műszaki és tudományos publikálás	bookdown, Everything curl., GitBook, Microsoft Learn
Prezentációkészítés	Marp, remark, reveal.js
Adattudományi eszközök	Project Jupyter
MI eszközök	ChatGPT (OpenAI), Gemini (Google)

XML bevezető

Mi az XML?

Általános célú jelölőnyelv. Születése az 1990-es évek második felében. Napjaink egyik meghatározó, az iparban széles körben használt technológiája.

Szűkebb értelemben: szintaxis strukturált dokumentumok ábrázolására, mely lehetővé teszi azok automatikus feldolgozását (elektronikus dokumentum formátum).

Tágabb értelemben: egy sereg közös töről fakadó specifikációt jelent, melyeket összefoglaló néven XML családnak neveznek.

Az XML és a HTML összehasonlítása

	XML	HTML
Címkekészlet	Nincs előre definiált	Előre definiáltat használ
Célja	Adatok leírása	Információ megjelenítés
Mire való?	Adatcsere formátumként használják	Egy prezentációs nyelv
		Tekinthető az XML egy speciális alkalmazásának (XHTML)

Az XML előnyei és hátrányai

Előnyök

- **Egyszerűség**, az XML dokumentumok közönséges szöveges állományok, melyeket akár a legegyszerűbb szövegszerkesztővel is létre lehet hozni.
- Nyíltság
- Gyártófüggetlenség
- Platformfüggetlenség
- Univerzális adatcsere formátum
- Kiterjedt infrastruktúra
- Az iparban de-facto szabvány

Hátrányok

- Bőbeszédű és nehézkesen használható szintaxis
- Nagy tárigény
- Bonyolultság, se szeri, se száma az XML-hez kötődő specifikációknak

Dokumentum központú és adatközpontú XML

Dokumentum központú XML

A dokumentumokat jelölésekkel megtűzdelt folyó szöveg alkotja. A dokumentumok szerkezete nagy változatosságot mutat. Lényeges az elemek sorrendje. Az ilyen dokumentumok tartalma elsősorban emberi

fogyasztásra szánt. Ilyen alkalmazás például az XHTML.

Adatközpontú XML

A dokumentumokat nagyszámú adatelem alkotja. Kevésbé véletlenszerű dokumentum-szerkezet. Az elemek sorrendje kevésbé lényeges. Az ilyen dokumentumok elsősorban gépi feldolgozásra szántak. Ilyen alkalmazás például az SVG.

XML 1.0

Leírja az XML dokumentumoknak nevezett objektumokat és részben előírja az ezeket feldolgozó programok viselkedését.

XML dokumentumok, jólformáltság

Olyan szöveges objektumok, melyek a szabvány előírásai szerint jólformáltak. **Fizikai és logikai szerkezetük van.** Fizikailag egyednek nevezett tárolási egységekből állnak. Logikailag deklarációkból, elemekből, megjegyzésekből, feldolgozási utasításokból és további szerkezeti alkotóelemekből állnak.

Jólformáltság

Egyetlenf felső szintű elem, az úgynevezett **gyökérelem** tartalmazza az összes többi elemet. Minden nyitó címkéhez tartozik egy megfelelő záró címke. Az elemek megfelelő módon egymásba ágyazottak, nem fedhetik egymást. Minden a dokumentumban hivatkozott elemzett egyed jólformált. Sok követelménynek - úgynevezett jólformáltsági megszorításnak - kell teljesülnie.

Elemek

Minden elemet nyitó és záró címke határol vagy egyetlen üres elem címke alkot.

```
<author>Sir Arthur Conan Doyle</author>
<message xml:lang="en">Hello, World!</message>

```

A nyitó, záró és üres címkében adott nevet elemtípusnak nevezzük. Jólformáltsági megszorítás, hogy a nyitó és záró címkében adott nevek meg kell, hogy egyezzenek. A nyitó és a záró címke fogja közre az elem tartalmát. Az elemekhez meg lehet adni attribútum-specifikációknak nevezett név-érték párokat. Az üres elem tartalom nélküli elem. Megadható közvetlenül egymást követő nyitó és záró címkével vagy üres elem címkével.

```
<elem></elem> <elem/>
```

Speciális karakterek (az & és < karakterek)

Az '&' és '<' karakterek ebben a formában kizárólag jelölő-határolóként, megjegyzésekben, feldolgozási utasításokban és CDATA-szakaszokban fordulhatnak elő.

- Helyettük minden egyéb helyen karakterhivatkozásokat, illetve az & és < egyedhivatkozásokat kell használni!
- A '>' karakter megadható az > egyedhivatkozással.

Hogy attribútumértékek is tartalmazhassanak aposztrófokat és idézőjeleket, az "'" és '"' karakterek megadhatók az ' és " egyedhivatkozásokkal.

Jelölők

Nyitó címke

```
<title xml:lang="en">  
<title>
```

Záró címke

```
</title>
```

Üres elem címke

```
<br/>, <hr />
```

Jólformáltsági megszorítás: a nyitó és üres elem címkében minden attribútumnév legfeljebb egyszer szerepelhet. Lényegtelen az attribútumok megadásának sorrendje.

Karakterhivatkozás

Szövegben, attribútum értékekben és literális egyed értékekben Unicode karakterek kifejezhetők az alábbi formájú karakterhivatkozásokkal:

```
&#nnnn; ahol nnnn a kódpontot ábrázoló decimális számjegysorozat  
&#xhhhh; ahol hhhh a kódpontot ábrázoló hexadecimális számjegysorozat
```

Egyedhivatkozás

Hivatkozás egy névvel azonosított egyed tartalmára.

- Hivatkozás elemzett általános egyedre: &név; - &
- Paraméteregyed-hivatkozás: %név; - %inline;

Megjegyzés

A dokumentumban bárhol szerepelhetnek más jelölőkön kívül. Az egyetlen kivétel a dokumentumtípus-deklaráció, melyben bizonyos helyeken előfordulhatnak.

```
<!-- Ez egy megjegyzés -->
```

Feldolgozási utasítás

Az alkalmazások számára tartalmaznak utasításokat.

```
<?xml-stylesheet type="text/css" href="style.css"?>
```

CDATA-szakasz határolók

Bárhol előfordulhatnak a dokumentumban, ahol előfordulhat karakteres adat. Olyan karaktereket tartalmazó szövegrészek levédésére szolgálnak, melyek egyébként jelölőként lennének tekintve. A CDATA-részben csak a ']]>' karakterlánc tekintett jelölőnek.

```
<![CDATA[if (0 < n && n <= 10)]]>
```

XML deklaráció

Az XML dokumentumoknak egy XML deklarációval ajánlott kezdődniük, mely meghatározza az XML használt verziószámát. A karakterkódolást meg kell adni, ha a használt kódolás nem az UTF-8 vagy az UTF-16, kivéve ha a kódolást egy magasabb szintű protokoll (például a HTTP) határozza meg.

```
<?xml version="1.0"?>  
<?xml version='1.0' encoding='UTF-8'?>
```

Dokumentumtípus-deklaráció

Dokumentumok egy osztályához egy nyelvtant meghatározó jelölő deklarációkat tartalmaz és/vagy ilyen deklarációkra mutat. Ezt a nyelvtant **dokumentumtípus-definíció**nak (vagy röviden DTD-nek) nevezik. A név a dokumentumtípus-deklarációban a gyökérelem elemtípusát írja elő.

Egy külső DTD alkészletnek nevezett, jelölő deklarációkat tartalmazó külső egyedre mutató dokumentumtípus-deklaráció:

```
<!DOCTYPE score-partwise SYSTEM "http://www.musicxml.org/dtds/partwise.dtd">  
<!DOCTYPE score-partwise SYSTEM "partwise.dtd">  
<!DOCTYPE score-partwise PUBLIC "-//Recordare//DTD MusicXML 3.1 Partwise//EN"  
"http://www.musicxml.org/dtds/partwise.dtd">
```

Jelölő deklarációkat - az úgynevezett **belső DTD alkészletet** - tartalmazó dokumentumtípus-deklaráció:

```
<!DOCTYPE message [  
  <!ELEMENT message (#PCDATA)>  
  <!ATTLIST message xml:lang CDATA #IMPLIED>  

```

Egy külső DTD alkészletre mutató dokumentumtípus-deklaráció, mely belső DTD alkészletet is tartalmaz:

```
<!DOCTYPE play SYSTEM "play.dtd" [  
  <!ENTITY r "Rosencrantz">  
  <!ENTITY g "Guildenstern">  
  <!ENTITY rag "&r; and &g;">  

```

Dokumentumtípus-definíció, érvényesség

Dokumentumtípus-definíció

Lehetővé teszi megszorítások előírását a dokumentum logikai szerkezetére, valamint tárolási egységek használatát támogatja. Jelölő deklarációkból áll.

A dokumentumtípus-deklarációban társítható egy dokumentumhoz. A dokumentumtípus-deklaráció mutathat egy külső DTD alkészletre, tartalmazhat egy belső DTD alkészletet, vagy teheti mindkettőt.

A két alkészlet együtt alkotja egy dokumentum DTD-jét. Ha a külső és belső alkészlet is van, akkor a belső alkészlet megelőzi a külső alkészletet. Ez lehetővé teszi a külső alkészletben lévő egyed- és attribútumlista-deklaráció felülírását.

Érvényesség

Egy XML dokumentum érvényes akkor, ha tartozik hozzá dokumentumtípus-deklaráció és a dokumentum eleget tesz a DTD által kifejezett megszorításoknak. A specifikáció pontosan előírja, hogy mely megszorítások kell, hogy teljesüljenek. Ezek az úgynevezett **érvényességi megszorítások**.

- A dokumentumtípus-deklarációban adott név meg kell, hogy egyezzen a gyökérelem elemtípusával.
- A dokumentum minden egyes eleme deklarált kell, hogy legyen a DTD-ben és az elem tartalma meg kell, hogy feleljen a deklarációnak.
- A dokumentum minden egyes attribútuma deklarált kell, hogy legyen a DTD-ben és az attribútum értéke meg kell, hogy feleljen a deklarációnak.
- Ha egy attribútum kötelezőnek deklarált a DTD-ben, akkor az elemtípus minden egyes előfordulásához explicit módon meg kell adni a dokumentumban.

Elemtípus-deklarációk

Egy elemtípus-deklaráció megszorítást ír elő az elem tartalmára. **Érvényességi megszorítás**: egy elemtípus nem deklarálható egynél többször.

Üres elem

Érvényességi megszorítás: az így deklarált elemeknek nem lehet tartalma.

```
<!ELEMENT br EMPTY>
...
<br/>
<br><br>
```

Elemtartalom

Egy elemtípus elemtartalmú, ha az ilyen típusú elemek csak elemgyermekeket tartalmazhatnak (karakteres adatot nem), melyek opcionálisan *whitespace* karakterekkel választhatók el.

A deklaráció egy **tartalommodell**t határoz meg, egy olyan egyszerű nyelvtant, mely az elemgyermek típusát és ezek megengedett sorrendjét szabályozza.

Tartalommodell megadásánál használható konstrukciók:

- Sorozat, mint például (street, city, zip, country)
- Alternatív lista, mint például (ul | ol | dl)

Az előfordulások számát szabályzó speciális karakterek:

- ?: nulla vagy egy (0, 1) előfordulás
- +: tetszőleges számú, de legalább egy (1, 2, 3, ...) előfordulás
- *: tetszőleges számú (0, 1, 2, ...) előfordulás

Ezekből a konstrukciókból összetett kifejezések alkothatók.

```
<!ELEMENT a (b,b)*> -> olyan a elem, ami páros  
számú b elemet tartalmaz  
<!ELEMENT a (b, (b,b)*)> -> olyan a elem, ami  
páratlan számú b elemet tartalmaz  
<!ELEMENT a (b, (b,b?)?)> -> olyan a elem, ami  
1, 2, vagy 3 b elemet tartalmaz
```

Deklaráció	The content of e	Az e elem érvényes példányai
<IELEMENT e (f)>	Pontosan egy f elem	<e><f></e>
<IELEMENT e (f?)>	0 vagy 1 f elem	<e></e> <e><f></e>
<IELEMENT e (f+)>	1 vagy több f elem	<e><f></e> <e><f><f></e> <e><f><f><f></e> ...
<IELEMENT e (f*)>	0 vagy több f elem	<e></e> <e><f></e> <e><f><f></e> ...
<IELEMENT e (f,g,h)>	Egy f, egy g és egy h elem, ebben a sorrendben	<e><f><g><h></e>
<IELEMENT e (f g h)>	Pontosan egy f, g vagy h elem	<e><f></e> <e><g></e> <e><h></e>

Érvényességi megszorítás: az így deklarált elemek tartalma

meg kell, hogy feleljen a deklaráció tartalommodelljének. Az első elemgyermek előtt, az elemgyermek
között és az utolsó elemgyermek után megengedettek *whitespace* karakterek, megjegyzések és feldolgozási
utasítások.

Előre definiált egyedek

Előre definiált egyedek speciális karakterekhez: amp('&'), lt('<'), gt('>'), apos(''), quot(''). Ezeket az egyedeket
minden XML feldolgozó fel kell, hogy ismerje, függetlenül attól, hogy deklaráltak-e vagy sem.

JavaScript / ECMAScript

Mi az ECMAScript/JavaScript?

ECMAScript

A JavaScript programozási nyelv szabványosítása. A jelenlegi aktuális a 15-ös számú kiadás: ECMAScript 2024 Language Specification.

JavaScript

A JavaScript kifejezést használják az ECMAScript különböző gyártók általi megvalósításaira.

JavaScript motor

Olyan szoftver, ami végrehajtja a JS kódot.

A fő JavaScript motorok

	SpiderMonkey	V8	JavaScriptCore	GraalVM Community Edition (CE)	Hermes	JerryScript
Programozási nyelv	C/C++	C++	C++	Java	C++	C
Licenc	Mozilla Public License 2.0	New BSD License	LGPLv2	GPLv2	MIT License	Apache License 2.0
Leírás	A Mozilla projekt JavaScript motorja.	A Chromium JavaScript motorja.	A WebKit renderelő motorhoz fejlesztett JavaScript motor.	Linux, macOS és Windows rendszerekhez áll rendelkezésre.	React Native alkalmazások futtatásához optimalizált JavaScript motor.	Pehelysúlyú JavaScript motor IoT eszközökre.

Mi a Node.js?

A V8 JavaScript motorra épülő JavaScript futtató környezet, melyet skálázható jálózeti alkalmazások létrehozásához terveztek.

- Licenc: MIT License
- Programozási nyelv: C++, JavaScript
- Platform: Linux, macOS, Windows

Lehetővé teszi a fejlesztők számára böngészőn kívül futó JavaScript alkalmazások létrehozását. Használható kliens oldali és szerver oldali alkalmazások fejlesztéséhez is. Csomag ökoszisztémája, az npm, a világ legnagyobb nyílt forrású könyvtár ökoszisztémája.

JSON

Mi a JSON?

Állománynév végződés: .json **IANA média típus:** application/json

A JSON és az ECMAScript összehasonlítása

Az ECMAScript 2019-től kezdve a JSON az ECMAScript szintaktikai részhalmaza.

A JSON és az XML összehasonlítása

A JSON az XML alternatívájaként használható adatcseréhez. Nagyjából ugyanazokat az előnyöket kínálja, mint az XML, azonban annak hátrányai nélkül.

A JSON és az XML közös jellemzői

- Egyszerűség (egyértelműen a JSON a nyerő)
- Az ember számára is könnyen írható és olvasható formátumok
- Szoftverek által könnyen generálható és feldolgozható formátumok (egyértelműen a JSON a nyerő)
- Interoperabilitás
- Nyíltság
- Önléíró adatábrázolás
- Univerzális adatcsere formátumok

Különbségek

A fő különbség az, hogy a JSON adat-orientált, az XML pedig dokumentum-orientált.

- Adatszerkezetek ábrázolásához a JSON tökéletes választás. Előnye az XML-hez képest, hogy kevésbé bőbeszédű.
- Dokumentum-középpontú alkalmazásokhoz az XML-t használjuk. Előnye a JSON-hoz képest, hogy kiterjeszthető, és hogy kiforrottabb infrastruktúra áll hozzá rendelkezésre (XML Schema, XSLT, XQuery)

Az XML és a JSON közötti konverzió

```
<movie>
  <title>The Dark Knight</title>
  <year>2008</year>
  <url>https://www.imdb.com/title/tt0468569/</url>
  <standalone>false</standalone>
</movie>
```

```
{
  "movie": {
    "title": "The Dark Knight",
    "year": 2008,
    "url": "https://www.imdb.com/title/tt0468569/",
    "standalone": false
  }
}
```

```
}  
}
```

```
<properties>  
  <property name="user.home">/home/jeszy</property>  
  <property name="user.name">jeszy</property>  
</properties>
```

```
{  
  "properties":{  
    "property": [  
      {  
        "@name": "user.home",  
        "#text": "/home/jeszy"  
      },  
      {  
        "@name": "user.name",  
        "#text": "jeszy"  
      }  
    ]  
  }  
}
```

Primitív típusok

Sztringek

Unicode karakterek sorozatai, melyeket idézőjelek (U+0022) határolnak. Bármely karaktert tartalmazhatják, azonban az alábbiakat csak levédve:

- idézőjel (U+0022)
- backslash (U+005C)
- vezérlő karakterek (U+0000 - U+001F)

Speciális karakterek megadásához rendelkezésre állnak a szokásos escape szekvenciák, mint például ", \, \t, \n, \r.

A BMP-hez tartozó Unicode karakterek megadhatóak \unnnn módon, ahol nnnn a karakterkód négy hexadecimális számjeggyel ábrázolva.

Számok

Nincs korlátozás a számok tartományára és pontosságára. A gyakorlatban célszerű szem előtt tartani az interoperabilitást. Például dupla pontosságú lebegőpontos számok használatára javasolt.

Pl.: 0; -1; 2.718282, 1E-12

Logikai értékek

null

Strukturált típusok

Tömbök

Tetszőleges számú érték rendezett sorozata (lehet üres). Az elemek tetszőleges típusúak lehetnek (akár tömbök is).

Pl.: [9, 14, 19]

Objektumok

Tetszőleges számú név-érték párból állnak. A név tetszőleges sztring, az érték tetszőleges JSON érték. A név-érték párokra a tag (member) elnevezést is használjuk.

RFC 8259

Olyan objektum interoperabilitás, melynél a tagok nevei egyediek. Eltérően viselkedhetnek az alkalmazások nem egyedi nevek esetén. Nem minden JSON elemző esetén állapítható meg a név-érték párok sorrendje.

```
{  
  "title": "Alien",  
  "year": 1979  
}
```

JSON szöveg különböző rendszerek közötti átvitelkor az UTF-8 karakterkódolást kell használni.

CSS

Mi a CSS?

Strukturált (pl. HTML, XML) dokumentumok megjelenítésének leírására szolgáló stíluslap nyelv. Szétválasztja a dokumentumok megjelenítési stílusát a dokumentumok tartalmától.

A CSS fejlesztése, CSS szintek

A W3C-n belül a CSS Munkacsoport fejleszti.

A szó hagyományos értelmében a CSS-nek nincsenek verziói, hanem szintjei vannak:

- CSS Level 1
- CSS Level 2
- CSS Level 3
 - Jelenleg is fejlesztés alatt áll.
 - Moduláris felépítésű:
 - Modulokra van bontva, ahol minden egyes modul a CSS egy részét definiálja.
 - A moduloknak is vannak szintjei: 1.szint: nincs megfelelője CSS2-ben, 3.szinttől: CSS2 létező lehetőségeit frissítő modulok.
 - A CSS modulok eltérő stabilitási szintűek.

A CSS minden egyes szintje az előzőn alapul, annak definícióit finomítja és új lehetőségeket vezet be.

CSS dobozmodell

A CSS egy fastruktúrájú dokumentumot kap, melyet egy rajzvászonon (például a képernyőn) jelenít meg egy olyan közbülső struktúrát, a dobozfát (box tree) előállítva, mely a megjelenített dokumentum formázási szerkezetét ábrázolja.

Minden egyes doboz a fában a dokumentum egy megfelelő elemét (vagy pseudo-elemét) ábrázolja térben és/vagy időben a rajzvászonon.

A CSS minden egyes elemhez nulla vagy több dobozt generál az elem display tulajdonsága által meghatározott módon. Egy elem általában egyetlen dobozt generál.

Szintaktikai elemek

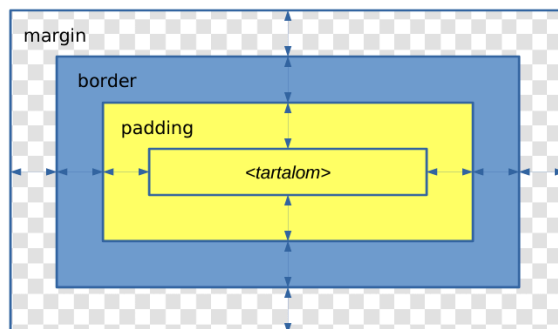
Karakterek

Az Unicode karakterkészlet használata.

Vezérlősorozatok

Unicode karakterek megadásához használhatunk \hhhhh formájú vezérlősorozatokat, ahol hhhhhh az Unicode karakter kódpontját ábrázoló legalább egy és legfeljebb 6 karakterből álló hexadecimális számjegysorozat.

- \9, \09, ..., \000009 a vízszintes tabulátor karaktert jelenti.



- \A9, \0A9, ..., \0000A9 a copyright szimbólumot (©) jelenti.

Speciális karakterek jelentésének elnyomásához használjuk a " karaktert. Például szükséges akkor, ha kiválasztóban pont karaktert tartalmazó elemnevet kell megadni.

- Például csak a given.name kiválasztóra illeszkedik a given.name nevű elem, a given.name kiválasztóra nem!

Megjegyzések

- A /* és */ határolók között lehet megadni megjegyzéseket.
- Példa: /* Style sheet for index.html */
- Tokeneken kívül bárhol megengedettek.
- Nem ágyazhatóak egymásba.

Deklarációs blokk

- '{' és '}' karakterek határolják, melyek között deklarációk egy listája kötelező.
- A deklarációk név:érték formájúak, ahol a tokenek előtt és után is megengedettek whitespace karakterek.
- A deklarációban név egy azonosító.
- A deklarációkat ';' karakterrel kell elválasztani.
- Az utolsó deklaráció után nem kötelező a ';' karakter.

at-szabályok

A stíluslap feldolgozását vezérlő speciális szabályok. Egy '@' karakterrel kezdődnek, melyet egy azonosító követ és ';' karakterrel vagy egy deklarációs blokkal végződnek.

- Példák: @charset, @import, @namespace, @media

Stílus szabályok

Egy kiválasztóból (vagy ';' karakterekkel elválasztott kiválasztókból) és egy az(oka)t követő deklarációs blokkból állnak.

Tulajdonságok

A CSS által definiált paraméterek, melyek révén a dokumentumok megjelenítése vezérelhető. A tulajdonságoknak neve és értéke van.

Összevont tulajdonság (shorthand property): Olyan tulajdonság, mely több CSS tulajdonság értékének egyidejű beállítására szolgál. Például a margin tulajdonság a margin-top, margin-right, margin-bottom és margin-left tulajdonságok értékét állítja be.

Stílus eredet

Különböző eredetű stíluslapok állhatnak rendelkezésre dokumentumok megjelenítéséhez.

Felhasználói ágenstől származó

A felhasználói ágensnek biztosítanak alapértelmezett stíluslapot. Például olyan stílus szabály tartalmazása, mely az em HTML elem megjelenítéséhez kurzív betűtípust ír elő.

Felhasználótól származó

A felhasználó megadhat saját stíluslapot adott dokumentum megjelenítéséhez.

Szerzőtől származó

HTML esetén a *link* fejléc elemmel adható meg a dokumentumhoz külső stíluslap.

```
<link rel="stylesheet" href="style.css">
```

Beágyazott stílus a style fejléc elemmel:

```
<style>
  h1, h2, h3, h4, h5, h6 {
    font-variant: small-caps
  }
</style>
```

XML esetén az xml-stylesheet feldolgozási utasítással adható meg a dokumentumhoz külső stíluslap. A feldolgozási utasítás a dokumentum gyökéreleme előtt kell, hogy szerepeljen.

```
<?xml-stylesheet type="text/css" href="style.css"?>
```

"Fontos" deklaráció (!important)

Egy deklarációt követheti a '!' token és az **important** kulcsszó. Egy ilyen deklaráció felülír bármely más közönséges deklarációt.

```
color: red !important
```

Alapértelmezésben a szerzői stíluslap szabályok nagyobb precedenciával bírnak, mint a felhasználói stíluslap szabályok. A szerzői és a felhasználói stíluslapok is tartalmazhatnak !important deklarációkat, ilyenkor ezek precedenciája megfordul: a felhasználói !important szabályok felülírják a szerzői !important szabályokat.

```
table.chessboard > tbody > tr:nth-child(odd) > td:nth-of-type(odd){
  background-color: white;
}
table.chessboard > tbody > tr:nth-child(odd) > td:nth-of-type(even) {
  background-color: lightgray;
}
table.chessboard > tbody > tr > td:hover {
  background-color: salmon !important;
}
```


Ebben az esetben, ha nem lenne ott a !important az utolsó szabálynál, akkor az sose lenne aktív, mivel a felső két szabály specifikussága nagyobb, mint a harmadiké.

A kaszkád

Kaszkádolás: konfliktusfeloldási mechanizmus arra az esetre, amikor egy elem/tulajdonság kombinációhoz különböző deklarációk állítanak be értéket.

Több különböző deklaráció szolgáltathatja egy tulajdonság értékét egy elemhez. Ezek a deklarációk különböző eredetűek is lehetnek. A **kaszkád** az a folyamat, melynek során meghatározásra kerül, hogy a vonatkozó deklarációk közül melyik határozza meg egy adott elem egy adott tulajdonságának értékét. A kaszkád az alábbi módon határozza meg egy adott elemhez egy adott tulajdonság kaszkádolt értékét:

- Meg kell határozni azokat a deklarációkat, melyek az adott elemhez az adott tulajdonság értékét szolgáltatják.
- Rendezzük a vonatkozó deklarációkat az eredetük szerint csökkenő "erőssorrendbe".
- Az azonos eredetű deklarációkat rendezzük specifikusság szerint csökkenő sorrendbe.
- A tulajdonság értékét a fenti sorrendben első deklaráció szolgáltatja.

Szabályok sorrendje

Lényeges lehet a szabályok sorrendje. Akkor számíthat a sorrend, ha egy elemre egynél több azonos specifikusságú stílus szabály vonatkozik. Ezek közül mindig a sorrendben utolsó a "legerősebb".

```
a:active {color: red} -> a = 0, b = 1, c = 1
a:hover {color: green} -> a = 0, b = 1, c = 1
a:visited {color: black} -> a = 0, b = 1, c = 1
a:link {color:blue} -> a = 0, b = 1, c = 1
```

Mivel itt a sorrendben utolsó a legerősebb, ezért az a művelet mikor rákattintunk / rávisszük az egeret sose fog teljesülni, mivel a másik két állapot egyike lesz látható. Ezért fontos a sorrend. A helyes sorrend:

```
a:visited {color: black} -> a = 0, b = 1, c = 1
a:link {color:blue} -> a = 0, b = 1, c = 1
a:hover {color: green} -> a = 0, b = 1, c = 1
a:active {color: red} -> a = 0, b = 1, c = 1
```

Öröklés

Öröklés, kezdőértékkadás: abban az esetben alkalmazásra kerülő mechanizmusok, amikor egy elem/tulajdonság kombinációhoz egy deklaráció sem állít be értéket.

Az öröklés a tulajdonságértékek továbbadását jelenti a szülő elemektől a gyermek elemekhez. Bizonyos tulajdonságok öröklöttek, ami azt jelenti, hogy az értékük öröklés révén kerül meghatározásra, feltéve, hogy a kaszkád nem eredményez egy értéket.

A specifikáció minden egyes tulajdonsághoz meghatározza, hogy öröklött-e

- **Öröklött:** font-family, font-style, font-variant, font-weight, font-size, font tulajdonságok
- **Nem öröklött:** margin-top, margin-bottom, margin-right, margin-left, margin és width tulajdonságok

Egy tulajdonság kaszkádolt értékeként az inherit kulcsszó öröklést kényszeríti ki.

```
<span style="color: red">
```

Az `<em>erő</em>` legyen veled!

```
</span>
```

Ebben az esetben, mivel az em HTML elemre nem vonatkozik color tulajdonság, így a span-tól öröklí.

```
<style>
```

```
  a:visited, a:link {  
    color: inherit;  
  }
```

```
</style>
```

```
<p style="color: green">
```

Click here: `<a href="https://www.w3.org/">W3C</a>`

```
</p>
```

Itt pedig az inherit kulcsszó miatt lesz a link zöld.

Hibakezelés

Ha a CSS-ben hiba fordul elő, akkor az elemző megpróbál abból úgy helyreállni, hogy csak a normál elemzéshez való visszatéréshez minimálisan szükséges tartalmat dobja el.

Minden egyes érvénytelen konstrukciót (deklarációt, stílus szabályt, at-szabályt) figyelmen kívül hagy a felhasználó ágens és folytatódik az elemzés.

- Ha egy stílus szabály kiválasztó listája egy érvénytelen kiválasztót tartalmaz, akkor a felhasználó ágens a teljes stílus szabály figyelmen kívül hagyja.
- Az érvénytelen, tehát figyelmen kívül hagyott deklarációk a DevTools-ban láthatók.

```
A
```

```
p {
```

```
  color: white
```

```
  background color: navy;
```

```
  font-family: sans-serif;
```

```
  unknown: 0;
```

```
  font-size: xxx;
```

```
}
```

stílus szabály gyakorlatilag ugyanaz, mint az alábbi:

```
p { font-family: sans-serif; }
```

Dobozok méretének meghatározása CSS-ben

box-sizing

A box-sizing CSS tulajdonság vezérli, hogyan kerül meghatározásra egy doboz szélessége és magassága. Azt határozza meg, hogy figyelembe kerül-e a keret és a belső margó (padding). A tulajdonság értéke *content-box* vagy *border-box*:

- **content-box:** a width és height CSS tulajdonságok a tartalom által elfoglalt (content area) méretét határozzák meg, azaz a keret és a belső margó nem kerül beszámításra.

```
div.box {  
  border: 10px solid black;  
  box-sizing: content-box;  
  height: 200px;  
  padding: 20px;  
  margin: 50px;  
  width: 300px;  
}  
Szélesség: 360px, Magasság: 260px
```

- **border-box:** a width és height CSS tulajdonságok a szemmel is látható méretet határozzák meg, melybe beleszámít a keret és a belső margó is.

```
div.box {  
  border: 10px solid black;  
  box-sizing: border-box;  
  height: 200px;  
  padding: 20px;  
  margin: 50px;  
  width: 300px;  
}  
Szélesség: 300px, Magasság: 200px
```

width és height

A width és height CSS tulajdonságok a dobozok **preferált szélességét és magasságát** határozzák meg.

min-width / min-height

A dobozok minimális szélességét/magasságát állítják be. Megakadályozzák, hogy a width/height tulajdonságok használt értéke kisebb legyen a min-width/min-height tulajdonságok előírt értékénél. Egy elem szélessége/magassága min-width/min-height értékére kerül beállításra, ha min-width/min-height értéke nagyobb mint max-width/max-height vagy width/height.

max-width / max-height

A dobozok maximális szélességét/magasságát állítják be. Megakadályozzák, hogy a width/height tulajdonságok használt értéke nagyobb legyen max-width/max-height előírt értékénél. A max-width/max-height tulajdonságok felülírják a width/height tulajdonságokat, azonban a min-width/min-height tulajdonságok felülírják a max-width/max-height tulajdonságokat.

CSS kiválasztók

A CSS kiválasztók mintaillesztésre szolgálnak. Meghatározzák, hogy egy szabály mely elemekre vonatkozik.

Kisbetű-nagybetű éréketlenség az ASCII tartományban (ekvivalensek az [a-z] és [A-Z] karakterek) az olyan részek kivételével, melyek nem a CSS hatálya alá esnek (elemnevek, attribútumnevek és -értékek).

Egyszerű kiválasztó

Egyetlen típus kiválasztó, általános kiválasztó, attribútum kiválasztó, osztály kiválasztó, ID-kiválasztó vagy pszeudo-osztály.

```
div - általános kiválasztó
[rel=stylesheet] - attribútum kiválasztó
.copyright - HTML osztály kiválasztó
#logo - ID kiválasztó
:lang(hu) - Pszeudo osztály kiválasztó
```

Kombinátor

Egy CSS-kiválasztó több egyszerű kiválasztót is tartalmazhat. Az egyszerű kiválasztók közé kombinátort kell beépíteni.

```
whitespace - leszármazott kombinátor
'>' - Gyermek kombinátor
'+' - Szomszéd testvér kombinátor
'~' - Általános kombinátor
```

Egyszerű kiválasztók sorozata

Egymást követő egyszerű kiválasztók, melyek között nincsenek kombinátorok. Típus kiválasztóval vagy általános kiválasztóval kezdődnek és nem tartalmaznak további típus kiválasztókat vagy általános kiválasztókat.

```
div
h2#status
link[rel=stylesheet][type=text/css]
p.copyright
*lang(hu) - *:univerzális kiválasztó
tr:nth-child(odd)
li:not(:last-child)
```

Kiválasztó

Egyszerű kiválasztók sorozatainak olyan sorozata, melyeket kombinátorok választanak el. Legalább egy sorozat alkotja. Pszeudo-elem at utolsó egyszerű kiválasztó sorozat végéhez adható hozzá, de legfeljebb csak egy!

```
p - 1 szekvencia bekezdés  
a img - olyan img elem ami leszármazottja a-nak  
h1 ~ table - ugyanahhoz a szülőhöz tartoznak  
div#main > h1 - h1 gyermeke a div#main-nek  
div > h1 + p  
q::before
```

Kombinátor

Leszármazott kombinátor

Egyszerű kiválasztók két sorozatát elválasztó whitespace.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a P Q kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, *melyek a P-re illeszkedő elem leszármazottai*.

```
thead th {  
    background-color: darkgrey;  
}
```

Gyermekek kombinátor

Egyszerű kiválasztók két sorozatát elválasztó '>' karakter.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a P > Q kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, *melyek a P-re illeszkedő elemek gyermekei*.

```
nav > div {  
    display: inline;  
}  
p > img:only-child {  
    margin-left: 0;  
}
```

Szomszéd testvér kombinátor

Egyszerű kiválasztók két sorozatát elválasztó '+' karakter.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a P + Q kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, *melyek a P-re illeszkedő elemet követnek közvetlenül a dokumentumban*.

- Az illeszkedő elemeknek ugyanaz kell, hogy legyen a szülője.
- Köztük megengedettek olyan konstrukciók, melyek nem elemek, például szöveg és megjegyzések, ezek figyelmen kívül hagyása.

```
h1 + p {  
    text-indent: 0;  
}
```

Általános testvér kombinátor (CSS3)

Egyszerű kiválasztók két sorozatát elválasztó '-' karakter.

Ha P és Q egyszerű kiválasztók két sorozata, akkor a $P \sim Q$ kiválasztóra a Q-ra illeszkedő olyan elemek illeszkednek, melyek a P-re illeszkedő elemet követnek (nem felétlenül) közvetlenül a dokumentumban.

- Az illeszkedő elemeknek ugyanaz kell, hogy legyen a szülője.

```
img ~ span {  
  color: red;  
}
```

Kiválasztók típusai

Típus kiválasztó

Egy CSS minősített név, a gyakorlatban tipikusan egy azonosító. A megfelelő nevű elemek illeszkednek rá.

```
p {color: red } - A p elemek színe legyen piros  
a { text-decoration: none } - Ha az elem link, akkor nem lesz aláhúzva
```

Általános kiválasztó

Általános kiválasztónak nevezzük a * formájú kiválasztót. Minden elem illeszkedik rá. Elhagyható olyan egyszerű kiválasztóból, mely további komponenseket is tartalmaz.

Ekvivalens kiválasztók

```
*#nav = #nav  
*.important = .important  
*[title] = [title]
```

Nem ajánlott elhagyni, az olvashatóság végett

```
div *:first-child = div :first-child  
div *: first-child != div:first-child
```

Attribútum kiválasztók

Az attribútum kiválasztókban érték azonosító vagy sztring. Tehát például a következők ekvivalensek:

```
[dir=rtl]  
[dir='rtl']  
[dir="rtl"]
```

Fajtái [att]: az att attribútummal rendelkező elemek illeszkednek rá

[att=érték]: olyan elemek illeszkednek rá, melyek att attribútumának értéke pontosan érték

```
style[type=italic] {  
    font-style: italic;  
}  
style[type=bold] {  
    font-weight: bold;  
}  
style[type=normal] {  
    font-style: normal;  
    font-weight: normal;  
}
```

Osztály kiválasztó

HTML dokumentumoknál a [class ~= érték] attribútum kiválasztó helyett használható az ekvivalens **.érték kiválasztó**.

div: Általános célú blokk, amit arra használnak, hogy egymástól elkülönítsék a tartalmat

centered: középre igazítható

margin-left:auto; - margin-right: auto; vízszintesen középre igazítás

```
div.centered {  
    margin-left: auto;  
    margin-right: auto;  
}
```

Az important azonosítójú elemek lesznek azok amiket elsődlegesként kezelnek. color: red; piros szín beállítása
text-decoration: underline; a szöveg aláhúzott

```
.important {  
    color: red;  
    text-decoration: underline;  
}
```

ID-kiválasztó

#azonosító formájú kiválasztó, az adott azonosítójú elem illeszkedik rá.

Az azonosítót egy ID típusú attribútum kell, hogy szolgáltatssa a dokumentumban. A dokumentum nyelvétől függ, hogy mi ennek az attribútumnak a neve, például a HTML-ben id.

```
div#main {  
    width: 50%;  
    margin-left: auto;  
    margin-right: auto;  
}
```

```
#footer {  
    text-align: center;
```

}

Pszeudo-osztályok

:azonosító vagy :azonosító(érték) formájú kiválasztó

Jellemzői:

- **Kisbetű-nagybetű érzéketlen**, általában kisbetű
- **Olyan kiválasztást tesznek lehetővé, mely a dokumentumon kívül információkon alapul vagy nem fejezhető ki a többi egyszerű kiválasztóval.**
- **Bizonyos pszeudo-osztályok egymást kölcsönösen kizáróak** (pl. :link és :visited)
- Bizonyos pszeudo-osztályok **dinamikusak**: ezek olyan pszeudo-osztályok, melyeket egy elem megszerezhet vagy elveszíthet, miközben a felhasználó interakcióban van a dokumentummal.
 - **A dinamikus pszeudo-osztályok két fajtája**: link pszeudo-osztályok és felhasználói akció pszeudo-osztályok.

Dinamikus pszeudo-osztályok

Link pszeudo-osztályok

:link: a felhasználó által még nem meglátogatott hiperhivatkozásokra vonatkozik.

```
a:link {  
  color: indigo;  
  text-decoration: none;  
}
```

:visited: a felhasználó által meglátogatott hiperhivatkozásokra vonatkozik

```
a:visited {  
  color: gray;  
  text-decoration: line-through;  
}
```

Felhasználói akció pszeudo-osztályok

:hover: a felhasználó által mutatóeszközzel kijelölt, de nem feltétlenül aktivált elemre vonatkozik.

```
a:hover {  
  border-width: medium;  
  border-style: solid;  
}
```

:active: a felhasználó által mutatóeszközzel aktivált elemre vonatkozik. Például az egérgomb lenyomása és felengedése között hatásos.

```
a:active {  
  font-weight: bolder;  
}
```



```
}
```

:focus: a fókusz birtokló elemre vonatkozik.

További pszeudo-osztályok

A :target pszeudo-osztály

Ha a dokumentum URI-ja tartalmaz erőforrásrész-azonosítót, akkor az erőforrásrész-azonosítónak megfelelő elem kiválasztása. Ha például `index.html#copyright` a dokumentum URI-ja, akkor a `div:target` kiválasztó a `copyright` azonosítójú `div` elemet választja ki.

A :lang(C) pszeudo-osztály:

A C nyelvű szöveget tartalmazó elemek illeszkednek rá, ahol C egy CSS azonosító (nyelvkód). Például:

:lang(en)

XML dokumentumokban a nyelvet az `xml:lang` attribútum határozza meg, HTML dokumentumokban a nyelv megadható a `lang` és az `xml:lang` attribútummal is. *A `lang` és `xml:lang` attribútum nem közvetlenül az elemen kell, hogy megjelenjen.*

```
A :lang(C) pszeudo-osztály használatakor magyar nyelvű idézet esetén:  
q:lang(hu){  
    quotes:""" "" " ">" "<";  
}
```

ahol a `q`, egy rövid idézet ami nem tör új sort. A `lang(hu)` jelöli a magyar nyelvet. A `quotes` mondja meg, hogy milyen határolók között legyen. A `"<"` és párja jelöli azt, hogy mire cserélje az idézőjeleket ha alaphoz van a szövegben.

:is() pszeudo-osztály

`:is(X)` formában adható meg, ahol az argumentum egy kiválasztó lista. `Matches-any` pseudo-class néven is ismert. Minden olyan elem illeszkedik rá, mely illeszkedik a lista argumentum valamely kiválasztójára. Hosszú kiválasztó listák tömörebb alakban való ábrázolásához hasznos.

A pszeduo-elemek nem érvényesek az `:is()`-en belül. Az `:is()` kiválasztó specifikussága az argumentum lista legnagyobb specifikusságú kiválasztójának specifikussága.

`:is()` pszeudo-osztály: ekvivalens például az alábbi két stíluslap szabály:

```
div:is(.note, .warning, .hint)::before{  
    /* ... */  
}  
  
div.note::before,  
div.warning::before,  
div.hint::before{  
    /* ... */  
}
```

A negáció pszeudo-osztály

:not(X) formában adható meg, ahol az argumentum egy olyan egyszerű kiválasztó, melyben nem megengedett a negáció pszeudo-osztály. Azok az elemek illeszkednek rá, melyek nem illeszkednek az argumentumra. Pl.: :not(.important), :not([title]), li:not(:last-child)

Szerkezeti pszeudo-osztályok

Amikor megállapításra kerül, egy elem helye a testvéreinek listájában, akkor csak az elemeket kell a listában figyelembe venni. Nem kell például figyelembe venni a szöveget, a megjegyzéseket és a feldolgozás utáni utasításokat. A listában az elemek számozása egytől egyig történik.

- **:root:** a dokumentum gyökérelemét választja ki
- **:only-child:** olyan elemeket választ ki, melyek szülőjének nincs más elemgyermek
- **:only-of-type:** olyan elemeket választ ki, melyek szülőjének nincs más az elemmel megegyező kifejtett nevű elemgyermek
- **:empty:** a szöveget és elemeket nem tartalmazó elemeket választja ki, melyek azonban tartalmazhatnak megjegyzéseket és feldolgozási utasításokat.
- **:nth-child(an+b):** az olyan elemeket választja ki, melyeknek $an+b-1$ megelőző elemtestvére van valamely nem negatív n -re.
- **:nth-last-child(an+b):** az olyan elemeket választja ki, melyeknek $an+b-1$ következő elemtestvére van valamely nem negatív n -re.
- **:nth-of-type(an+b):** az olyan elemeket választja ki, melyeknek valamely nem negatív n -re $an+b-1$ olyan megelőző elemtestvére van, melyek kifejtett neve megegyezik az elem kifejtett nevével.
- **:nth-last-of-type(an+b):** az olyan elemeket választja ki, melyeknek valamely nem negatív n -re $an+b-1$ olyan következő elemtestvére van, melyek kifejtett neve megegyezik az elem kifejtett nevével.
- **:first-child:** azt jelenti, mint :nth-child(1)
- **:last-child:** azt jelenti, mint :nth-last-child(1)
- **:first-of-type:** azt jelenti, mint :nth-of-type(1)
- **:last** Argumentumokat meg lehet adni röviden:
 - $an+0 \rightarrow an$
 - $0n+b \rightarrow b$
 - Negatív b esetén $an-b$ módon kell megadni az argumentumot, nem megengedett $an+-b$
 - A $2n$ argumentum megadható *even* módon, a $2n+1$ argumentum pedig *odd* módon.

Pszeudo-elemek

Lehetővé teszik a dokumentumok olyan részeinek kiválasztását, melyek más módon nem hozzáférhetők. Minden kiválasztóban legfeljebb egy pszeudo-elem megengedett. Az alábbi pszeudo-elemek állnak rendelkezésre:

- ::first-line
- ::first-letter
- ::before, ::after A pszeudo-osztályoktól való megkülönböztetés miatt ':' helyett '::' karaktereket használnak a pszeudo-elemekhez a CSS3.

::first-line

Egy elem első formázott sorát ábrázolja.

```
p::first-line{
  text-decoration: underline
}
```

::first-letter

Egy elem első betű vagy számjegy karakterét ábrázolja, ha azt ben előzi meg más tartalom, például kép. Vonatkozik az első betű vagy számjegy karaktert megelőző vagy követő írásjelekre is.

```
p::first-letter {
  font-size: 2em;
  font-weight: bold;
}
```

::before, ::after

A ::before és ::after lehetővé teszik egy elem tartalmához generált tartalom hozzáadását.

```
div.proof::before{
  content: "Proof: ";
  font-weight: bold;
}

div.proof::after{
  content: "\220E" /* End of proof */
}
```

Specifikusság, a specifikusság meghatározása

Kiválasztókhöz és deklarációkhoz specifikusság meghatározása. A specifikusság egy háromelemű (a, b, c) vektor, ahol a, b és c nemnegatív egészek. A vektorok rendezése lexikografikusan történik.

Specifikusság meghatározása:

- **a:** a kiválasztóban előforduló ID-kiválasztók száma
- **b:** a kiválasztóban előforduló attribútum kiválasztók és pszeudo-osztályok száma
 - A negáció pszeudo-osztályt figyelmen kívül kell hagyni, azonban az argumentumát nem!
- **c:** a kiválasztóban előforduló típus kiválasztók és pszeudo-elemek száma.

Deklarációk specifikussága: Egy deklaráció specifikussága megegyezik a tartalmazó szabály kiválasztójának specifikusságával. Szabályhoz nem tartozó (HTML-ben a style attribútum értékeként adott) deklaráció specifikussága nagyobb minden kiválasztóénál.

CSS előfeldolgozók

CSS előfeldolgozó fogalma

A CSS előfeldolgozók olyan programok, melyek CSS kódot generálnak a CSS írására szolgáló saját szintaxisukból. A CSS-t új lehetőségekkel egészítik ki, mint például változók, egymásba ágyazás, függvények, mixinek.

CSS előfeldolgozók közös jellemzői

Egysoros megjegyzések

Az egysoros megjegyzések // kezdetűek és a sor végéig tartanak. A fordítás során nem generálnak CSS-t.

@import

A CSS előfeldolgozók a böngészőktől eltérően kezelik az @import szabályokat. A sima CSS importokhoz a böngészőnek az oldal renderelésekor kell HTTP kéréseket végrehajtania. A CSS előfeldolgozók az importokat a fordítás során kezelik.

Sass

Az állomány kiterjesztés: .sass / .scss. Történeti okokból a Sass azonosítóiban, mint például a változó-, függvény-, és mixin-nevekben, a kötőjelek és aláhúzójelek azonosként vannak kezelve. Ez azt jelenti, hogy például \$font-size és \$font_size is ugyanarra a változóra hivatkozik.

Az SCSS szintaxis

A CSS szintaxis egy kiterjesztése.

```
$size: 2em;

table.chessboard {
  td {
    height: $size;
    width: $size;
  }
}
```

Főbb lehetőségek

Innen hiányzik a változók!

Stílus szabályok egymásba ágyazása

Az egymásba ágyazással csoportosítani lehet a kapcsolódó stílus szabályokat, melyek kiválasztói ugyanazt a közös komponenst tartalmazzák.

	1D kiválasztó szó	2D kiválasztó szó	3D kiválasztó szó
Kiválasztó	a	b	c
* általános kiválasztó	0	0	0
div elem kiválasztó	0	0	1
p::first-letter elem kiválasztó pseudó-elem	0	0	2
a img elem kiválasztó	0	0	2
h1 + p elem kiválasztó kombinátor elem kiválasztó	0	0	2
div[class = nav] szó kiválasztó elem kiválasztó	0	1	1
div.nav szó kiválasztó elem kiválasztó	0	1	1
#main *:lang(en) ID kiválasztó pseudo-kiválasztó	1	1	0

Az egymásba ágyazás a stílus szabályokat szervezi, így az olvashatóságot és karbantarthatóságot javítja.

Egymásba ágyazás nélküli kód:

```
article {  
    background-color: blue;  
}  
article a {  
    color: green;  
}  
article > h2 {  
    text-decoration: underline;  
}  
article + p {  
    font-size: smaller;  
}
```

Egymásba ágyazott kód:

```
article{  
    background-color: blue;  
    a {  
        color: green;  
    }  
    > h2 {  
        text-decoration: underline;  
    }  
    + p {  
        font-size: smaller;  
    }  
}
```

Mixinek

A mixinek lehetővéteszik CSS deklarációk újrafelhasználás céljából történő csoportosítását. Akár argumentumot is kaphatnak.

```
@mixin rounded-border {  
    border: thin solid black;  
    border-radius: 0.25 em;  
}  
nav {  
    @include rounded-border;  
    background-color: azure;  
}  
p {  
    @include rounded-border;  
    background-color: pink;  
}
```

@extend

Az egyik kiválasztónak örökölnie kell egy másik kiválasztó stílusát.

```
.error:hover {  
  background-color: #fee;  
}  
  
.error--serious {  
  @extend .error;  
  border-width: 3px;  
}
```

Reszponzív webdizájn

Mi a részponzív webdizájn?

A részponzív webdizájn (RWD) egy webfejlesztési megközelítés olyan weblapok készítéséhez, melyek optimálisan néznek ki és viselkednek egy egész sor eszközön, a mobiltelefonoktól kezdve az asztali monitorokig, függetlenül attól, hogy mi az ablak-/képernyőméret vagy a tájolás. A részponzív weboldalak dinamikusan ahhoz az eszközhöz igazodnak, melyen megjelenítésre kerülnek.

Weboldal elrendezések:

Statikus/rögzített: abszolút mértékegységben (többnyire pixelben) kifejezett rögzített szélességet használó elrendezés.

Folyékony (fluid/liquid): relatív mértékegységben kifejezett szélességet használó elrendezés.

Adaptív: statikus elrendezések egy sorozatként tekinthető. Média lekérdezések révén különböző statikus elrendezéseket határoz meg különböző nézetablak szélességeihez.

Reszponzív: folyékony elrendezések egy sorozatként tekinthető. Média lekérdezések révén különböző folyékony elrendezéseket határoz meg különböző nézetablak szélességeihez.

Pixel

CSS pixelek: A CSS-ben használt mértékegységek, amelyek a referencia pixelhez igazodnak.

Fizikai pixelek: A képernyő tényleges pixelei, amelyek száma és mérete változhat az eszköztől függően.

Referencia pixel: A referencia pixel az a látószög, mely alatt egy pixel látszik egy 96dpi pixelsűrűségű eszközön egy kar távolságról nézve.

A CSS abszolút és relatív hosszúság mértékegységei, nézetablak mértékegységek

A CSS relatív hosszúság mértékegységei

- **em:** a font-size tulajdonság az elemhez tartozó számított értékét jelenti.
- **ex:** az elemhez tartozó betűkészlet x-height értékét jelenti (a font-size tulajdonság esetén a szülő elemhez tartozó x-height értékét). Azért nevezik így, mert gyakran a betűkészlet x karakterének magasságával egyezik meg értéke.
- **rem:** az em egység a gyökéremlemhez tartozó számított értékét jelenti.

A nézetablak-arány mértékegységek

- **vw:** a nézetablak szélességének 1%-ával egyenlő.
- **vh:** a nézetablak magasságának 1%-ával egyenlő.
- **vmin:** a vw és vh közül a kisebbel egyenlő.
- **vmax:** a vw és vh a nagyobbbal egyenlő.

A CSS abszolút hosszúság mértékegységei

- **cm** (centiméter): 1 cm = 96 px / 2.54
- **mm** (miliméter): 1 mm = 1 cm / 10
- **Q** (negyvened miliméter) 1 Q = 1 cm / 40
- **in** (inch) 1 in = 2.54 cm = 96 px
- **pc** (pica) 1 pc = 1 in / 6
- **pt** (pont) 1 pt = 1 in / 72
- **px** (pixel) 1 px = 1 in / 96

Média lekérdezések

Egy média lekérdezés egy módszer a felhasználói ágens vagy eszköz bizonyos jellemzőinek vizsgálatára, melyen a dokumentum megjelenítésre kerül. Stíluslapok feltételes alkalmazására szolgálnak. Majdnem mindig függetlenek a megjelenített dokumentumtól, csak "külső" információktól függenek. **Média lekérdezések az alábbi nyelvekben használhatóak:**

CSS-ben

```
@import "screen.css" screen;  
@import "print.css" print;
```

Az opcioné kis média lekérdezés lista a feltételeket fejezi ki, melyek teljesülése esetén alkalmazható az @import. Ha a média lekérdezls nem illeszkeedik, akkor az importált stíluslap szabályai nem lesznek alkalmazva.

```
@media screen {  
    * {  
        font-family: sans-serif;  
    }  
}
```

A **@media** szabály egy olyan feltételes csoportszabály, melynek feltétele egy média lekérdezés. A feltételes csoportszabályok egy feltételt társítanak más CSS szabályok egy csoportjához.

HTML-ben

A media attribútumot, mely a következő elemeken adható meg:

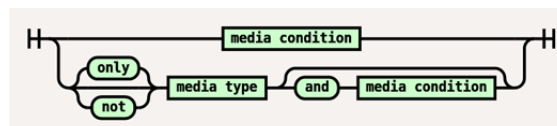
- link, meta, source, style

```
<link rel="stylesheet" media="screen" href="screen.css">  
<link rel="stylesheet" media="print" href="print.css">
```

Média lekérdezés szintaxis

Egy média lekérdezés egy opcionális módosítóból, egy opcionális média típusból és egy média feltételből áll.

Több média lekérdezés egy vesszővel elválasztott média lekérdezés listává kombinálható.



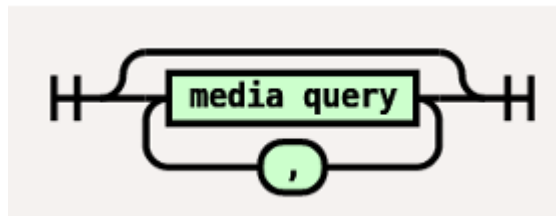
Média lekérdezés kiértékelés

Egy média lekérdezés egy logikai kifejezés, mely igaz vagy hamis. Egy média lekérdezés igaz, ha:

- Meg van adva a média típus, akkor az illeszkedik annak az eszköznek a média típusára, ahol a felhasználói ágens fut.

Egy média lekérdezés lista igaz, ha valamely komponens média lekérdezés igaz, és csak akkor hamis ha minden komponens média lekérdezés hamis.

```
@media screen, print { /*...*/ }
```



Média típusok

Egy média típus felhasználói ágens eszközök egy széles csoportja, melyeken megjeleníthető egy dokumentum. A következő média típusok definiáltak média lekérdezésben való felhasználásra:

- **all:** az összes eszközre illeszkedik.
- **print:** a nyomtató eszközökre illeszkedik és az olyan eszközökre, melyek célja egy nyomtatott megjelenítés visszaadása, mint például egy dokumentum nyomtatási képét mutató böngésző.
- **screen:** a print-re nem illeszkedő összes eszközre illeszkedik.

A felhasználói ágensnek érvényesként kell, hogy felismerjék ezeket a média típusokat, de semmit sem illeszthetnek rájuk.

Média jellemzők (csak a width, height és orientation)

Egy média jellemző (media feature) egy, a média típusoknál finomabb teszt, mely a felhasználó ágens vagy megjelenítő eszköz egy bizonyos jellemzőjét vizsgálja. A média jellemzők szintaktikailag a CSS tulajdonságokra hasonlítanak: **egy jellemző névből, egy kettőspontból és egy értékből állnak.**

A min- és max- előtagok (min-width, max-width, min-height, max-height)

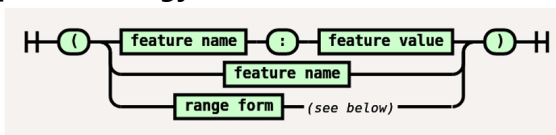
Egy tartomány típusú médai jellemző a tartomány kontextusban történő kiértékelése helyett írható közönséges média jellemzőként, de a neve előtt a min- vagy max- előtaggal.

- **min-** előtag használata egy jellemző néven ekvivalnes a $\geq$ operátor használatával

```
(min-height: 600px) = (height >= 600)
```

- **max-** előtag használata egy jellemző néven ekvivalnes a $\leq$ operátor használatával

```
(max-width: 40em) = (width <= 40em)
```



Diszkrét típusú média jellemzőkhöz nem adható meg min- vagy max-előtag.

Média jellemzők kombinálása (not, and, or)

Bármely média jellemző negálható elé **not** helyezésével, mint például **not (color)** Két vagy több média jellemző összefűzhető közük **and/or** helyezésével.

```
(width < 600px ) and (height < 600px)
(update: slow) or (hover: none)
```

Média feltételek csoportosíthatók zárójelek közé zárással, mint például not ((color) or (hover)). Nem megengedett egy médai lekérdezés ugyanazon "szintjén" keverni az and/or/not logikai műveleteket.

A látható és az elrendezési nézetablak, a viewport meta címke

Látható nézetablak (visual viewport): az oldal a kijelzőn látható része.

Elrendezési nézetablak (layout viewport): a nézetablak, melybe a böngésző kirajzol egy oldalt.

A **viewport meta** címkét abból a célból vezették be, hogy lehetővé tegye a szerző számára az (elrendezési) nézetablak méretének és a kezdeti nagyítási arány közvetlen megadását.

Mobile-first és desktop-first dizájn

Mobile-first

A Mobile-first megközelítés azt jelenti, hogy a weboldalt először mobil eszközökre tervezzük, majd a nagyobb képernyők irányába haladva bővítjük és finomítjuk a dizájnt.

Kezdeti fókuszt: Kis képernyőméretek, például okostelefonok. **Tartalom prioritása :** A legfontosabb információkat helyezzük előtérbe, mivel kis képernyő áll rendelkezésre

Desktop-first

A Desktop-first megközelítés során a weboldalt először nagy képernyőkre (asztali számítógépek, laptopok) tervezzük, majd a kisebb kijelzők felé haladva egyszerűsítjük a dizájnt.

Kezdeti fókuszt : Nagyobb képernyők (pl. 1024 pixel feletti szélességek). **Bonyolultabb elrendezések:** A nagyobb hely miatt több vizuális elem, komplexebb navigációk. **Media queryk:** Az alapstílusok az asztali nézethez igazodnak, és kisebb képernyőkhöz írunk felül

Folyékony rácsok: a flexbox és a rács elrendezés

A RWD egy rugalmas rács-alapú elrendezést igényel, ahol a "rács" kifejezés nagyon általánosan értendő.

Flexbox elrendezés

Felhasználói felületek tervezéséhez optimalizált CSS dobozmodell. A flex elrendezési modellben egy flex konténer gyermekei bármely irányban elrendezhetők és rugalmasan változtathatják a méretüket: nyúlhatnak a nem használt hely kitöltéséhez vagy összezsugorodhatnak. Könnyen kezelhető a gyermekek vízszintes és függőleges igazítása is. Ilyen dobozok egymásba ágyazása felhasználható elrendezések két dimenzióban történő létrehozásához.

```
body {  
    margin: 0;  
}  
  
.cards {  
    display: flex;  
    flex-wrap: wrap; /*ennek köszönhetően reszponzív az elrendezés*/  
    gap: 1em;  
    margin: 1em;  
}
```

Rács elrendezés

Egy felhasználói felületek tervezéséhez optimalizált kétdimenziós rács-alapú elrendezési rendszert határoz meg. A rács elrendezési modellben egy rács konténer gyermekei tetszőleges rekeszekbe helyezhetők egy előre definiált rugalmas vagy rögzített méretű elrendezési rácson. A rács elrendezés lehetővé teszi a dobozok és tartalmuk méretezésének és elhelyezésének vezérlését.

```
body {  
    margin: 0;  
}  
  
.cards {  
    display: grid;  
    gap: 1em;  
    grid-template-columns: repeat(auto-fit, 2.5in);  
    margin: 1em;  
}
```

A WEB jelölő nyelvei

HTML, SVG, MathML, Markdown

Mi a HTML?

A HTML a Web elsődleges leíró nyelve. [...] egy szemantikai leíró nyelv és a kapcsolódó szemantikai szintű alkalmazásprogramozási interfészek a Weben elérhető oldalak készítéséhez, melyek a statikus dokumentumoktól a dinamikus alkalmazásokig terjednek.

HTML elemek

Az elemeknek, attribútumoknak és attribútumértékeknek meghatározott jelentése (szemantikája) van. Pl. ol -> rendezett lista, lang -> a tartalom nyelve.

Prezentációs jelölő használata: Rontja a hozzáférhetőséget, magasabb a karbantartási költsége, nagyobbá teszi a dokumentum méretét. Not good.... **csak a style attribútum és a style elem maradt meg!!!**

Elem	HTML 4.01	HTML5
b	Félkövér betű	Kulcsszavak
i	Kurzív betű	Hangnembeli változás
hr	Vízszintes választóvonal	Témaváltás
small	Kisebbségi betűméret	Lapszéli megjegyzés
s	Áthúzás	Pontatlan szöveg
u	Aláhúzás	Artikulálatlan jelölés (pl. hibás szöveg)

A specifikációban definiált minden egyes elemnek van egy tartalommodellje (az elem szükséges tartalmának egy leírása). Egy HTML elem tartalma meg kell, hogy feleljen a tartalommodelljében leírt követelményeknek. Minden egyes HTML elem nullva vagy több kategóriába sorolható az alábbiak közül:

Idegen elemek (foreign elements): A MathML és SVG névterekbe tartozó elemek.

Az összes HTML elem elérhető itt:

<https://html.spec.whatwg.org/multipage/indices.html#element-3>



Globális attribútumok

id

class

A szerzők az elemek kiterjesztéséhez használhatják a class attribútumot, gyakorlatilag saját elemeket létrehozva úgy, hogy közben a legalkalmasabb létező HTML elemet használják. A HTML elemeken megadott class attribútum értéke olyan tokenek egy szóközzel elválasztott listája, melyek azokat a különféle osztályokat ábrázolják, melyekhez az elem tartozik.

```
<p class="author"> ... </p>
<p class="note"> ... </p>
```

lang

style

egyéni adat attribútumok

Egy egyéni adat attribútum egy olyan attribútum, melynek neve a data- karakterlánccal kezdődik, melyet legalább egy karakter követ a kötőjel után. Olyan egyéni adatok, állapot, annotációk és más hasonló az oldal vagy alkalmazás számára történő privát tárolására szolgálnak, melyekhez nincsenek megfelelőbb elemek vagy attribútumok. Minden HTML elemhez tetszőleges számú egyéni adat attribútum adható meg tetszőleges értékekkel.

```
<form>
  <p>
    <label for="house">Ház:</label> <select name="house" id="house" required>
      <option value="">--Válassz egy lehetőséget--</option>
      <option value="gryffindor" data-color-primary="red" data-color-
secondary="gold">Griffendél</option>
      <!-- ... -->
    </select>
  </p>
</form>
```

HTML szintaxisok

A HTML egy dokumentum leírására szolgáló absztrakt nyelvet határoz meg, az ezt az absztrakt nyelvet használó erőforrások átviteléhez pedig két konkrét szintaxist.

HTML szintaxis

- Bár nagyon hasonlít az SGML-hez és az XML-hez, egy külön nyelv saját feldolgozási szabályokkal.
- Kompatibilis a legtöbb ősi böngészővel.
- Fájlkiterjesztés: .html, .htm
- Média típus: text/html

Követelező a dokumentumtípus-deklaráció.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Sample Page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <p>Hello, World!</p>
```

```
</body>
</html>
```

További adatok

- **Speciális karakterek:** Az elem szövege nem tartalmazhat '<' és '&' karaktert.
- Az elem- és attribútumnevek kisbetű-nagybetű érzéketlenek
- **Nem idézett attribútumérték szintaxis:** Abban az esetben ha az = jel után nem rakunk szóközt, akkor megadható az attribútum "" nélkül.

```
<input value="yes">
<input value=yes>
```

- **Logikai attribútumok:** Egy logikai attribútum jelenléte egy elemen az igaz értéket ábrázolja, hiánya pedig a hamis értéket.
- **Üres (void) érték:** Csak nyitó címkéje van, tilos lezárni. PL.: br, img, input, link, meta, stb.
- **Idegen elemek:** Ebben az esetben vagy van külön lezáró címkéje vagy önlezáró nyitó címkéje van.

```
<circle cx=50 cy=50 r=50></circle>
<circle cx=50 cy=50 r=50 />
```

- **Opcionális címkék:** Bizonyos elemek nyitó és záró címkéje elhagyható.
 - Elhagyható egy li elem záró címkéje, ha az elemet közvetlenül egy másik li elem követi, vagy nincs több tartalom a szülő elemben.

```
<ul>
  <li>Apple
  <li>Banana
  <li>Cherry
</ul>
```

- Elhagyható a html elem nyitócímkéje, ha elsőként nem egy megjegyzést tartalmaz.
- Nem támogatottak a névtér-deklarációk, még idegen elemekhez sem.
- CDATA-szakaszok csak idegen tartalomban (MathML vagy SVG) használhatóak.

XML szintaxis

- Az XML 1.0 és a Namespaces in XML 1.0 szabványokon alapuló szintaxis.
- Nem határoz meg további szintaktikai követelményeket az XML-hez az előírtakon túl.
- XHTML szintaxisnak is nevezik.
- Fájlkiterjesztése: .xhtml, .xht
- Média típus: application/xhtml+xml

```
<?xml version="1.0" encoding="UTF-8"?>
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en">
  <head>
    <title>SamplePage</title>
    <link rel="stylesheet" href="style.css"/>
```

```
</head>
<body>
  <p>Hello,World!</p>
</body>
</html>
```

Document Object Model (DOM)

Egy DOM fa egy dokumentum memóriabeli ábrázolása. A DOM egy alkalmazásprogramozási interfész (API) dokumentumok (főleg HTML és XML dokumentumok) eléréséhez és manipulálásához.

Minden dokumentumot egy fa ábrázol, mely csomópontokból (Document, DocumentType, Element, Text, ProcessingInstruction, Comment) áll. Minden egyes csomópontot egy API-val rendelkező objektum ábrázol, így tehát manipulálható. A DOM interfészek Web IDL-ben kerülnek leírásra.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Sample Page</title>
</head>
<body>
  <p>Hello, World!</p>
  <!-- This is a comment -->
</body>
</html>
```

```
{ DOCTYPE: html
  html lang="en"
  head
    #text: ""
    title
      #text: Sample Page
    #text: ""
  body
    #text: ""
    p
      #text: Hello, World!
    #comment: This is a comment
    #text: ""
```

- a WEB IDL egy interfészleíró nyelv, mely böngészőkben implementálható interfészek leírására szolgál
- a HTML specifikáció a HTML elemek ábrázolásához a DOM interfészeket kiterjesztő további interfészeket határoz meg
- a HTML implementációk megfelelési kritériumai is DOM műveletekkel vannak meghatározva
- egy DOM fa szkriptekből manipulálható az oldalon

A HTML dokumentumtípus-deklaráció

A HTML szintaxisban a dokumentumtípus-deklaráció szakszerű, melynek célja mindössze annak biztosítása, hogy a dokumentum megjelenítése a szabványos módon történjen.

Az XML szintaxisban tetszőleges dokumentumtípus-deklaráció használható, megadása nem is kötelező.

Böngészők megjelenítési módjai: kompatibilis mód, szabványos mód

A renderelő motorok az alábbi módokban jelenítenek meg HTML dokumentumokat:

Kompatibilis mód (quirks mode)

Régi böngészők viselkedésének utánzása (emulálása) az aktuális webszabványokat sértő módon régi weboldalak megjelenítéséhez.

Szabványos mód

A weboldalak megjelenítése az aktuális webszabványoknak megfelelően.

Hibakezelés

A HTML korábbi verzióitól eltérően az aktuális specifikáció az érvényes dokumentumok feldolgozása mellett bizonyos szintű részletességgel az is előírja, hogy hogyan kell feldolgozni az érvénytelen dokumentumokat. A webböngészők valójában nagyon elnézőek a hibákkal szemben, automatikusan kijavítják a nem érvényes tartalmat.

```
<html>
  <foo>
    <p bar class=> Does it <b>work,<i>or</b>not</i>?
  <
  </body>
</html>
```

Megfelelési ellenőrző vagy más néven érvényesítő (conformance checker, validator): Egy olyan szoftvereszköz, mely ellenőrzi, hogy egy HTML dokumentum teljesíti-e a HTML specifikáció megfelelési követelményeit.

Képek a CSS-ben és HTML-ben

A webes adatforgalom jelentős részét jelentik a képek (40%).

A főbb képformátumok

A HTML szabvány nem követeli meg a böngészőktől, hogy támogassák bármely képformátumot. Azonban a böngészők eltérő formátumokat támogathatnak.

JPEG

A JPEG egy veszteséges tömörítési eljárás, melyet főleg fotografikus képek kódolására használnak.

- Fejlesztő: Joint Photographic Experts Group
- Webhely: <https://jpeg.org>
- Média típus: image/jpeg
- Fájlkiterjesztés: .jpeg, .jpg
- A JPEG tömörítésű képadatok tárolására szolgáló formátum neve JFIF.
- Valamennyi főbb böngésző natív módon támogatja a JPEG 1-et.

PNG

A Portable Network Graphics (PNG) egy nyílt veszteségmentes raszteres képformátum, mely támogatja az alfa csatornát (átlátszóságot).

- Fejlesztő: W3C
- Webhely: <https://www.w3.org/Graphics/PNG/>
- Média típus: image/png
- Fájlkiterjesztés: .png
- Eredetileg azért tervezték, hogy szabadalmi védettség mentes alternatívát nyújtson a GIF formátumhoz.
- Valamennyi főbb böngésző natív módon támogatja

APNG: a Mozilla által javasolt egy kiterjesztése animált képekhez.

- Média típus: image/apng
- Fájlkiterjesztés: .apng

WebP

Egy nyílt raszteres képformátum mely támogatja a veszteséges és veszteségmentes tömörítést is, valamint az alfa csatornát (átlátszóságot) és animációt.

- Fejlesztő: Google
- Webhely: <https://developers.google.com/speed/webp>
- Média típus: image/webp
- Fájlkiterjesztés: .webp
- Hálózaton keresztül történő gyors képátvitelre optimalizálták
- Valamennyi főbb böngésző natív módon támogatja

AVIF

Egy nyílt raszteres képformátum mely támogatja a veszteséges és veszteségmentes tömörítést is, valamint az alfa csatornát (átlátszóságot) és animációt.

- Fejlesztő: Alliance for Open Media
- Webhely: <https://aomedia.org/specifications/avif/>
- Média típus: image/avif
- Fájlkiterjesztés: .avif
- Csupán az utóbbi néhány évben került be a főbb böngészőkbe az AVIF támogatás.

SVG

Egy nyílt képformátum kétdimenziós vektorgrafika XML-ben történő leírásához.

- Fejlesztő: W3C
- Webhely: <https://www.w3.org/Graphics/SVG/>
- Média típus: image/svg+xml
- Fájlkiterjesztés: .svg
- Támogatja az interaktív grafikát és az animációt is.
- Valamennyi főbb böngésző natív módon támogatja

Az HTML elem

Az elem egy képet ágyaz be egy HTML dokumentumba. Egy olyan helyettesített elem, mely display tulajdonságának értéke inline alapértelmezés szerint. A típusától függően egy képnek lehet belső (intrinsic) szélessége és magassága.

Kötelező attribútumok

- **src**: a képerőforrás URI-ját tartalmazza
- **alt**: egy helyettesítőszöveget tartalmaz

Opcionális attribútumok

- **height/width**: egy mértékegység nélküli egész szám, mely a kép belső magasságát/szélességét adja meg pixelben.
- **loading**: azt jelzi, hogy a böngésző hogyan töltsen be a képet, értéke eager vagy lazy

Az alt attribútum

Egy helyettesítő szöveget tartalmaz a képhez. Akkor jelenik meg ha:

- Ha a böngésző szöveges
- Ha a kép formátuma nem támogatott / hálózati hiba
- Ha a felhasználó úgy dönt nem kellenek neki képek

Az **alt** attribútum értéke egy világos és tömör helyettesítő szöveget kell, hogy szolgáltatson a kép tartalmához. Abban az esetben, ha üres sztringre állítjuk be, akkor a kép nem fontos a tartalomhoz.

```
<p> Ma találtam egy cicát az ajtónk előtt:  
    
</p>
```

A height/width attribútumok

A kép belső (intrinsic) magasságát/szélességét adják meg pixelben. A szélesség és magasság ismerete lehetővé teszi a böngésző számára, hogy még a kép betöltése előtt kiszámítsa a képarányt (aspect ratio). Ezen képarány alapján kerül a kép számára lefoglalásra a megjelenítéshez szükséges hely.

Az elrendezés eltolódás (layout shift) miatt is fontos megadni ezeket az attribútumokat, hiszen így betöltéskor le van foglalva a hely és nem fog random később betöltődni a kép, ezzel összezavarva az olvasást.

A HTTP haladó lehetőségei

Űrlap adatok továbbítása a GET és a POST metódusokkal, különbségeik

GET metódus: Az adatokat az URL-hez csatolt lekérdezési paraméterekkel küldi el. Nem érzékeny adatokhoz alkalmas, és hasznos, ha az URL-t könyvjelzőzni vagy megosztani szeretnénk. Hosszúsági korlátai vannak, és kevésbé biztonságos.

POST metódus: Az adatokat a kérés törzsében küldi el, így biztonságosabb az érzékeny információkhoz. Nincs méretkorlátja, és nem látható az URL-ben, így alkalmasabb űrlapok beküldésére.

Sütik

Egy név-érték pár és kapcsolódó metaadatok (attribútumok), melyeket egy eredet szerver egy válasz **Set-Cookie** fejlécmezőjében küld a felhasználói ágensnek.

A felhasználói ágens a további kérésekben a név-érték párt a **Cookie** fejlécmezőben küldi vissza az eredet szervernek.

Felhasználások:

- Munkamenet kezelés
- Testreszabás
- Webes követés

Amikor a felhasználói ágens egy **Set-Cookie** fejlécmezőt kap, eltárolja az attribútumaival együtt.

A továbbiakban, amikor a felhasználói ágens egy HTTP kérést hajt végre, a **Cookie** fejlécmezőbe illeszti az alkalmazható, nem lejárt sütiket.

Ha a felhasználói ágens olyan új sütit kap, melynek neve, valamint **Domain és Path** attribútuma megegyezik egy már tárolt sütivel, akkor az új sütire cseréli ki a korábbi.

Süti attribútumok

A sütik maximális élettartamát jelző attribútumok:

- **Expires:** a süti lejáratának dátumát és idejét adja meg.
- **Max-Age:** azt adja meg, hogy hány másodperc múlva jár le a süti.
- Ha a sütinek Max-Age és Expires attribútuma is van, akkor a Max-Age élvez elsőbbséget.

Domain: Meghatározza, hogy a süti mely szervereknek lesz elküldve. Ha a szerver kihagyja az attribútumot, akkor a felhasználói ágens csak az eredet szervernek küldi vissza a sütit.

Path: A süti hatáskörét adott útvonalakra korlátozza. Ha a szerver kihagyja az attribútumot, a felhasználói ágens a kért URI útvonal komponensének "könyvtárát" használja alapértelmezett értéként.

Secure: A süti hatáskörének biztonságos csatornákra korlátozása. Egy Secure attribútummal rendelkező sütit a felhasználói ágens csak akkor tesz bele egy kérésbe, ha annak átvitele biztonságos csatornán keresztül történik.

HttpOnly: HTTP kérésekre korlátozza a süti hatáskörét. Arra utasítja a felhasználói ágenszt, hogy ne tegye a sütit elérhetővé kliens oldali API-k számára.

Sütik kezelése

Felhasználói ágenseknek: Törölniük kell a lejárt sütit. Az aktuális munkamenet végén törölniük kell az összes nem perzisztens sütit. Ajánlott a felhasználók számára lehetővé tenni a tárolt süti kezelését. Ajánlott a felhasználók számára lehetővé tenni a süti letiltását.

A felhasználói ágensek korlátokat szabnak a tárolható süti számára és méretére. A specifikáció az alábbi minimális képességeket ajánlja:

- Sütinként legalább 4096 bájt
- Tartományonként legalább 50 süti
- Összesen legalább 3000 süti

Harmadik féltől származó süti

Ezek a süti különösen problémásak. Egy HTML oldal megjelenítése során egy felhasználói ágens gyakran kér le erőforrásokat más szerverektől. Ezek a harmadik félnek számító szerverek sütit használnak a felhasználó követésére még akkor is, ha a felhasználó közvetlenül soha nem látogatja meg őket. Ezeket a sütit *cross-site* sütiknek is nevezik.

Webes követés

Egy adott felhasználó több különböző kontextuson keresztüli tevékenységével kapcsolatos adatgyűjtés és az ebből a tevékenységből származtatott adatok megőrzése, felhasználása vagy megosztása azon kontextuson kívül, melyben a tevékenység történik.

Alapjául szolgáló információk

IP-cím, Süti, ETag fejléczmező, Eszköz ujjlenyomat (operációs rendszer, képernyőfelbontás, telepített betűkészletek, ...)

a Referer fejléc-mező, védekezés a követés ellen

Lehetővé teszi a felhasználói ágens számára, hogy megadja azt az erőforrást azonosító URI-hivatkozást, melyből a cél URI származik.

Referer: https://www.w3.org/

Potenciális információkat nyújthat a felhasználói böngészési előzményeiről, mely egy biztonsági kockázat.

Nem ajánlott a Referer fejléczmező küldése akkor, ha a hivatkozó erőforrás elérésre biztonságos protokollon keresztül történt és a kérés céljának eredete eltér a hivatkozó erőforrásától, hacsak nem engedi meg kifejezetten a Referer mező küldését a hivatkozó erőforrás.

Tilos a Referer fejléczmező küldése nem biztonságos HTTP kérésben, ha a mezőértékben jelzett oldal biztonságos protokollon keresztül érkezett.

A fejléczmező tiltása csak Firefoxban lehetséges.

Eredet

Az eredetek alapvetőek a webes biztonság szempontjából. Feltételezhető, hogy megbízik egymásban két olyan aktor a weben, melyeknek ugyanaz az eredete. A különböző eredetű aktorokat egymással szemben potenciálisan ellenségesnek tekintjük, elkülöníteni ajánlott őket.

Egy erőforrás eredetét az eléréséhez használt URI sémá, hoszt és port komponense határozza meg.

Same-origin policy

Egy azt korlátozó kritikus biztonsági mechanizmus, hogy egy adott eredet által betöltött dokumentum vagy szkript hogyan kerülhet kapcsolatba egy másik eredetű erőforrással. Általában tilos egy adott eredetű kód számára egy másik eredetű kód elérése.

HTTP/2

A HTTP szemantikájának egy optimalizált kifejezőmódja. Cél a hálózati erőforrások hatékonyabb használata és a végfelhasználó által megfigyelhető késleltetési idő csökkentése.

Az egyik fő cél, hogy lehetővé tegye a kliensek számára, hogy csupán egy kapcsolatot kelljen fenntartaniuk egy szerverhez.

Ugyanazokat a metódusokat, állapotkódokat, fejlécmezőket és URI sémákat használja, mint a HTTP/1.1

Bevált gyakorlatok a késleltetési idő csökkentésére

- **Beágyazás:** képek közvetlen beágyazása CSS stíluslapokba
- **Spriting:** több kép kombinálása egyetlen képállományba
- **Sharding:** a tartalom elosztatása több szerveren
- **Content Delivery Network (CDN):** A tartalomszolgáltató hálózat (CDN) összekapcsolt szerverek hálózata, amely felgyorsítja a weboldalak betöltését nagy adatforgalmú alkalmazások esetén. A CDN jelentése tartalomszolgáltató hálózat vagy tartalomelosztó hálózat.
- **Összefűzés:** több CSS stíluslap és JavaScript állomány összefűzése
- **Kicsinyítés:** felesleges karakterek eltávolítása CSS stíluslapokból és JavaScript állományokból anélkül, hogy az erőforrás a böngésző általi feldolgozása módosulna

A HTTP/2 újdonságai

Multiplexelés: megvalósítás adatfolyamok használatával

Forgalomvezérlés és rangsorolás: a multiplexelt adatfolyamok hatékony használatát biztosító mechanizmusok.

Szerver push: lehetővé teszi a szervernek, hogy spekulált módon küldjön olyan adatokat egy kliensnek, melyekre annak előreláthatólag szüksége lesz.

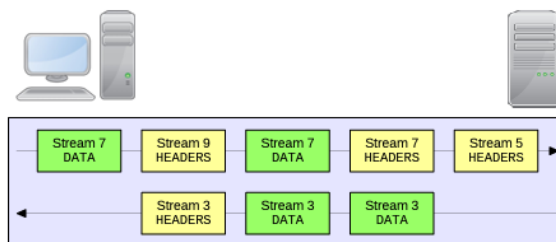
Bináris protokoll: az üzenetek hatékonyabb feldolgozását teszi lehetővé az üzenetek bináris formálására.

Fejlécmezők tömörítése (HPACK)

Üzenet multiplexelés

Keretek

Minden keret egy 9 oktett méretű fejléccel kezdődik, melyet egy változó hosszú adatrész (payload) követ.



- **Hossz:** az adatrész hosszát szolgáltató 24-bites előjel nélküli egész
- **Típus:** a keret 8 biten ábrázolt típusa, mely meghatározza a keret felépítését és szemantikáját
- **Jelzők:** 8 biten ábrázolt logikai jelzők, melyek jelentése a keret típusától függ.

- **R:** fenntartott célú 0 értékű bit, melynek nincs definiált jelentése.
- **Adatfolyam azonosító:** egy adatfolyamot azonosító 31-bites előjel nélküli egész.

Fajtai

Kód	Típus	Funkció
0x0	DATA	Üzenet payload átvitele.
0x1	HEADERS	Adatfolyam megnyitása és egy fejléc blokk töredék továbbítása
0x2	PRIORITY	Adatfolyam prioritásának előírása
0x3	RST_STREAM	Adatfolyam azonnali megszüntetése
0x4	SETTINGS	Konfigurációs paraméterek továbbítása, vétel nyugtázása.
0x5	PUSH_PROMISE	Szerver push megvalósítása: a másik végpont előzetes értesítése a küldő által létrehozni kívánt adatfolyamokról.
0x6	PING	Minimális körbefutási idő (RRT) mérése, tétlen (idle) állapotú adatfolyam működőképességének megállapítása.
0x7	GOAWAY	Kapcsolat lezárásának kezdeményezése vagy kapcsolati hiba jelzése.
0x8	WINDOW_UPDATE	Forgalomvezérlés megvalósítása.
0x9	CONTINUATION	Fejléc blokk töredékek sorozatának folytatása.

Adatfolyamok jellemzőik, azonosításuk

Egyetlen HTTP/2 kapcsolatot több egyidejűleg nyitott adatfolyamot tartalmazhat. Az adatfolyamokat egyoldalúan (a másik féllel való egyeztetés nélkül) hozhatja létre a kliens vagy a szerver, de megosztva használhatják őket. Az adatfolyamok mindkét végpont lezárhatja. Lényeges, hogy milyen sorrendben kerülnek elküldésre a keretek egy adatfolyamon. Az adatfolyamokat egy előjel nélküli egész szám azonosítja.

Azonosításuk

Az adatfolyam azonosító egy 31-bites előjel nélküli egész, melyet az adatfolyamot létrehozó végpont rendel az adatfolyamhoz.

- A kliens által megnyitot adatfolyamokat páratlan számok azonosítják.
- A szerver által megnyitott adatfolyamokat páros számok azonosítják.
- A 0x00 adatfolyam azonosítót a kapcsolatot vezérlő üzenetekhez használják.
- Egy újonnan létrehozott adatfolyam azonosítója nagyobb kell, hogy legyen a kezdeményező végpont által létrehozott vagy fenntartott adatfolyam azonosítók mindegyikénél.
- Az adatfolyam azonosítók nem újrafelhasználhatóak.
- Hosszú élettartamú kapcsolatoknál kimerülhet a rendelkezésre álló adatfolyam azonosítók tartománya.

HTTP/2 kérés/válasz váltás

Egy kliens minden egyes HTTP kérést egy új adatfolyamon küld el a szervernek, melyhez egy előzőleg nem használt adatfolyam azonosítót használ. A szerver ugyanezen az adatfolyamon küldi vissza a választ. A kérés/válasz váltás során az adatfolyam teljesen "elhasználódik". A választ záró keret lezárja az adatfolyamot.

Fejlécmezők, pseudo-fejlécmezők

A fejlécmezők nevét kisbetűssé kell alakítani a HTTP/2 szerinti kódolásuk előtt.

Egy fejlécmező kétféle módon ábrázolható a kódolásban:

- **Indexként:** a statikus vagy dinamikus indextábla egy bejegyzésére hivatkozó indexként.
- **Literálisan:** a fejlécmező nevének és értékének megadásával.
 - A fejlécmező neve ábrázolható literálisan vagy valamelyik indextábla egy bejegyzésére hivatkozó indexként.
 - A fejlécmező értékének ábrázolása literálisan történik.

Fejlécmező nevének és értékének literális ábrázolása történhet közvetlenül vagy egy statikus Huffman-kóddal.

Pszeudo-fejlécmezők

A HTTP/1.x üzenetek kezdősorában adott információk ábrázolásához a HTTP/2 ':' karakterrel kezdődő speciális pszeudo-fejlécmezőket használ. **A pszeudo-fejlécmezők nem HTTP fejlécmezők.**

Kérés pszeudo-fejlécmezők

- **:method:** a HTTP metódust tartalmazza
- **:scheme:** a cél URI séma részét tartalmazza
- **:authority:** a cél URI autoritás részét tartalmazza, a Host fejlécmező megfelelője. HTTP/2 kéréseket generáló kliensek számára az :authority pszeudo-fejlécmező használata ajánlott a Host fejlécmező helyett
- **:path:** a cél URI útvonal és lekérdezés részét tartalmazza. Szerver-szintű OPTIONS kéréseknél '*' az értéke.

Válasz pszeduo-fejlécmező

- **:status:** a HTTP állapotkódot hordozza, minden válasznak tartalmaznia kell.

A pszeudo-fejlécmezők meg kell, hogy előzzék a rendes fejlécmezőket. *CONNECT* kérések kivételével minden HTTP/2 kérés pontosan egy érvényes értéket kell, hogy tartalmazzon a *:method*, *:scheme*, és *:path* pszeudo-fejlécmezőkhöz.

A HTTP/2 nem teszi lehetővé a kérésben a protokoll továbbítását, valamint a válaszban a protokoll verzió és az indok frázis továbbítását.

HTTP3

Mi a HTTP/3?

A HTTP legutóbbi fő verziója, mely a QUIC átviteli protokollon alapul. A HTTP szemantika leképezése a QUIC átviteli protokollra.

Mi a QUIC?

Az UDP-re épülő biztonságos, általános célú átviteli protokoll. Kiejtése: quick